University of Hertfordshire UH

PROJECT REPORT

BEng (Hons) in Aerospace Engineering with Space Technology

Name **Dharshini Subramani**

Supervisor Dr. Pouria Sarhadi

Simulation of Satellite Attitude Control Using Deep Reinforcement Learning

20th March 2026

BACHELOR OF ENGINEERING DEGREE/DEGREE WITH HONOURS IN AEROSPACE ENGINEERING WITH SPACE TECHNOLOGY

Department of Engineering
School of Physics, Engineering and Computer Science
University of Hertfordshire

SIMULATION OF SATELLITE ATTITUDE CONTROL USING DEEP REINFORCEMENT LEARNING

Report by
Dharshini Subramani

Supervisor
Dr.Pouria Sarhadi

Date
20th MARCH 2026

DECLARATION STATEMENT

I certify that the work submitted is my own and that any material derived or quoted from the published or unpublished work of other persons has been duly acknowledged (ref. UPR AS/C/6.1, Appendix I, Section 2 – Section on cheating and plagiarism).

I confirm that any study conducted as part of my project that involved human participants has been approved in accordance with UH Ethical regulations and the protocol number(s) is/are:

N/A

Student Full Name: DHARSHINI SUBRAMANI

Student Registration Number: 22090321

Signed: ……Dharshini Subramani……

Date: ……20/03/2026……

ABSTRACT

Satellites often need extensive attitude control to orient themselves to required position and direction (such as pointing at Earth) with precision. As space missions are energy-critical, there was also an additional priority of consuming less energy as possible for attitude stabilization and orientation. Traditional control methods such as Proportional – Integral-Derivative (PID) controllers are widely employed in conjunction with on-off actuators for attitude control in satellite, but due to their fixed parameter tuning policy, which hindered their adaptability to non-linear dynamics, they struggle in handling important parameter variations. Therefore, this technique is being replaced by Machine Learning control techniques such as Deep Reinforcement Learning. This project investigates the use of Deep Reinforcement Learning as an intelligent substitute for single-axis satellite attitude control, implemented and evaluated through simulation in MATLAB and Simulink.

This project comprises of design, implementation and comparison of four controllers: a PID controller with relay block, a PID controller with both relay and dead zone activation, a Deep Deterministic Policy Gradient controller with continuous torque output and a Deep Q-Network controller whose discrete on-off actuation control matches the real thruster behaviour. All controllers are tested against five test scenarios which reflect realistic spacecraft conditions. These test scenarios are nominal step response, recovery from large initial angle, external torque disturbance simulation, inertia variation and detumbling. The satellite dynamics are based on Euler's rotational equations of motion for a rigid body with moment of inertia J = 11.4 kg.m^2 .

For nominal attitude tracking, through comparison and analysis, it was found that the DDPG controller achieves the fastest settling time of 3.2 seconds with 2.9% overshoot, and in terms of on-off actuation, DQN controller settles in 5.9 seconds with minimal overshoot and reduced number of discrete thruster pulses used. In the context of external disturbances, PID controllers dominate as they eliminate the steady state error of approximately 0.1 rad through integral action which persists in the DRL controllers under constant added external torque. Compared to PID with just relay, PID with both relay and deadzone gives the best performance as the deadzone action reduces chattering which lowers fuel consumption.

For large initial angle recovery, the DQN controller outperforms both DDPG and PID methods in which the satellite was able to recover and settle in a stable manner without any oscillations. When inertia of the satellite was modified from 11.4 to 1000 kg.m^2 (~ 88 * 11.4 kg.m^2) , the DQN controller acheives partial attitude control without retuning while the DDPG controller diverges completely. Therefore, in the context of parameter uncertainity, the learning policy of discrete DRL techniques adapt bettter than continuous DRL. When satellite detumbling scenario was simulated, the DQN controller was found to be successfully stabilising the satellite within 14.9 seconds while the PID controller diverges completely.

These findings demonstrate that, while PID still has advantages in disturbance rejection for nominal conditions, DRL-based controllers, particularly DQN with on-off actuation, outperform standard PID approaches in nonlinear and uncertain spacecraft operation scenarios. To improve DRL robustness, future research should address disturbance rejection constraints during training with enhanced observations and inertia randomisation.

ACKNOWLEDGEMENTS

I'd like to thank my supervisor, Dr. Pouria Sarhadi, for his direction and support during the project's progress.

I'd also like to thank the University of Hertfordshire for giving access to the MATLAB and Simulink software tools necessary to finish the simulations.

Finally, I'd like to express my gratitude to my family and friends for their support throughout the execution of this project.

TABLE OF CONTENTS

DECLARATION STATEMENT iii
ABSTRACT iv
ACKNOWLEDGEMENTS vi
TABLE OF CONTENTS vii
1. LIST OF FIGURES ix
GLOSSARY x
2. INTRODUCTION 1
2.1 Background 1
2.2 PROBLEM STATEMENT 1
2.1.3 PROJECT AIMS AND OBJECTIVES 2
2.1.4 Report Structure 2
3. LITERATURE REVIEW 3
3.1 Traditional Attitude Control Methods 3
3.2 Reinforcement Learning for Control 3
3.3 Deep Reinforcement Learning for Spacecraft Attitude Control 4
3.4 Research Gap and Project Motivation 5
3.5 Summary 5
4. METHODOLOGY 6
4.1 Satellite Dynamics Model 6
4.2 TRADITIONAL CONTROLLERS 6
4.2.1 PID controller with Relay Block 6
4.2.2 PID Controller with Relay and Deadzone Block 7
4.3 DDPG Continuous Controller 8
4.3.1 Algorithm Overview 8
4.3.2 Observation and Action Spaces 9
4.3.3 Actor and Critic Network Architecture 9
4.3.4 Reward Function 10
4.3.5 Training Configuration 11
4.4 DQN On-Off Controller 11
4.4.1 Algorithm Overview 11
4.4.2 Observation and Action Space 12
4.4.3 Network Architecture 12
4.4.4 Training improvements 12
4.4.5 Reward Function 13
4.4.6 Training Configuration 13
4.5 Test Case Definitions 13
4.6 Performance Metrics 14
5. Results and Discussion 15

5.1 Traditional Controllers (PID Baseline) 15
5.1.1 PID with Relay 15
5.1.2 PID with Relay and Dead Zone 19
5.2 DDPG Continuous Controller 23
5.3 DQN On-Off Controller 27
5.4 Comparative Analysis 32
5.4.1 Performance Comparison 32
5.4.2 Discussion 33
6. Conclusion 35
7. Project Management 36
7.1 Planned v/s Actual progress 36
7.2 Reflection on Project Management 36
7.3 Risk Assessment 37
REFERENCES 38
BIBLIOGRAPHY 39
APPENDIX A 40
APPENDIX B 47

1. LIST OF FIGURES

Figure 1 PID Controller model with Relay block. Source: Author 7
Figure 2 PID Controller Simulink Model with Relay and Dead zone. Source: Author 8
Figure 3 Schematic Diagram of RL operation process. Source: Qiu et al. (2023) 9
Figure 4 Schematic Diagram of the DDPG Architecture. Source: Qiu et al. (2023) 9
Figure 5 Critic network architecture. Source: Author 10
Figure 6 PID with relay nominal step response plots 15
Figure 7 PID with Relay case 2 response plots. 16
Figure 8 External disturbance injection (adapted from Sarhadi, 2025, pers. comm.) 17
Figure 9 PID with relay Case 3 response plots (Generated by MATLAB) 17
Figure 10 PID with Relay Case 4 Response Plots 18
Figure 11 PID with Relay Case 5 response plots 19
Figure 12 PID-RDZ Case 1 Response Plots 20
Figure 13 PID-RDZ Case 2 Response Plots 20
Figure 14 PID-RDZ Case 3 Response Plots 21
Figure 15 PID-RDZ Case 4 response plots 22
Figure 16 PID-RDZ Case 5 Response Plots 22
Figure 17 DDPG Controller - Case 1 Plots 23
Figure 18 DDPG Controller - Case 2 Plots 24
Figure 19 DDPG Controller Case 3 Plots 25
Figure 20 DDPG Controller-Case 4 Plots 25
Figure 21 DDPG Controller - Case 5 plots 26
Figure 22 DQN Controller Case 1 plots 27
Figure 23 DQN Controller Case 2 plots 28
Figure 24 DQN Controller Case 2 plots after adjustments 29
Figure 25 DQN Controller Case 3 plots 30
Figure 26 DQN Controller Case 4 plots 30
Figure 27 DQN Controller Case 5 plots 31

GLOSSARY

ADCS – Attitude Determination and Control System

PID – Proportional Integral Derivative Controller

RL – Reinforcement Learning

DRL – Deep Reinforcement Learning

DDPG – Deep Deterministic Policy Gradient

DQN- Deep Q Network

PID-RDZ – Proportional-Integral-Derivative with Relay and Dead zone

2. INTRODUCTION

2.1 Background

In recent times, satellites have been playing an increasingly important role in applications such as communications and weather forecasting, even in missions such as Earth observation or deep space exploration. In any satellite mission, one of the important objectives of the satellite is to able to orient itself precisely and maintain the same position, rejecting external disturbances and adapting to any changing operating conditions. This function is handled by one of the most critical subsystems of the satellite, called as the Attitude Control System (ACS). It is responsible for analysing the satellite's current orientation, calculating the difference between its current and desired position and applying corrective torques trhough actuators such as Reaction wheels to bring the satellite to its target orientation.

Traditional attitude control methods such as the Proportional-Integral-Derivative (PID) controller have been widely accepted in the industry due to its predictable behaviour and design simplicity. However, the continuous nature of PID control signals clashes with the discrete torque outputs of the actuators in practice. Modulation techniques such as the Pulse-Width Pusle-Frequency (PWPF) modulator and deadzone are used to train the PID controllers in adapting to on-off thruster commands. (Khosravi and Sarhadi, 2016)

Though these controllers have been the industry standard due to their mathematical simplicity, the mathematical models they have been based on are rigid, therefore they are not easily adaptable to parameter variations, such as changes in inertia due to fuel depletion, unless nalually re-tuned again. Therefore, if there are external disturbances or extreme changes in the operating conditions, the controller performance may degrade over time. In addition to these limitations, the PID controllers also struggle with stabilising the satellite in non-linear scenarios such as detumbling operations. However recent advances in machine learning have brought forward Reinforcement Learning (RL) as an intelligent substitute for PID controllers as a RL agent learns through interactions with the environment, therefore it does not rely on predefined control laws or mathematical models. (De Sa Marques, 2021)

2.2 PROBLEM STATEMENT

Modern spacecraft missions are becoming more complex, and with the growing complexity, the attitude controllers must adapt to the new demands and needs. These controllers must now handle any type of dynamics both linear and non-linear, along with other factors such as parameter uncertainities and unexpected disturbances. These controllers must also be able to track and use fuel efficiently as spacecraft missions are energy critical.

More adaptive and robust control methods have been proposed in literature as alternatives, but they require significant mathematical effort and possess high technical risk as even after development, they might not reach the stability targets required for specific cases. Deep Reinforcement Learning (DRL), a branch of machine learning, is being explored as a flexible alternative to the traditional

controllers. In DRL, the control agent learns control and stabilization techniques through a policy which is gained through direct interactions with the environment, eliminating the need for a predefined mathematical control model. (Pérez-Muñoz et al., 2025)

While DRL has demonstrated its superiority in continuous attitude control, its application to on-off thruster systems, which represent the real spacecraft actuators used, has been not widely studied. This project attempts to provide a direct comparative evaluation of DRL algorithms against traditional PID- based controllers across spacecraft test scenarios such as detumbling, inertia variation, etc.

2.1.3 PROJECT AIMS AND OBJECTIVES

This project's main goal is to use MATLAB to design and simulate Deep Reinforcement Learning-based control algorithms for a satellite attitude dynamics model and evaluate how well they perform in comparison to conventional PID-based controllers under various operating scenarios.

The specific objectives are as follows:

- To develop a mathematical model of a sateliite attitude dynamics using Euler's rotational equations of motion as the foundation and to implement it as a simulation plant in MATLAB.
- To develop and implement two traditional baseline controllers, a PID with relay block and another one is a PID controller with both relay and deadzone and to test their performance across 5 spacecraft scenarios.
- To design and train a DDPG continuous controller, under ideal continuous actuator conditions and to be evaluated acorss five spacecraft test scenarios.
- To design and train a DQN controller, matching the real discrete on-off thruster behaviour and to be tested across five scenarios
- To compare the controllers' performances across key metrics such as percentage of overshoot, settling time, steady state error and control effort and to draw conclusions on the advantages and disadvantages of DRL over traditional control methods and its use in real-time attitude control.

2.1.4 Report Structure

The remainder of the report is organised as follows. Chapter 2 presents a review of the relevant literature, covering traditional attitude control methods, foundation of Reinforcement Learning and applications of DRL in spacecraft operations. Chapter 3 describes the methodology of both satellite dynamics and the control methods. Chapter 4 presents the results for all the controllers across five test scenarios and the comparative analysis of the four controllers. Chapter 5 presents the conclusions of the project and gives a summary of the key findings, with any identified limitations and areas for future improvements.

3. LITERATURE REVIEW

3.1 Traditional Attitude Control Methods

Spacecraft attitude control has been in active research over decades with active pursuit for controllers which are adaptable and possess human-level performance. But, inspite of this, advanced PID controllers remain the industry standard as they have proven to be successful in controlling real systems and because of their simple design and predictavle behaviour under linear conditions. However, these traditional methods rely on mathematical models therefore they struggle when satellite faces non-linear dynamics, parameter variations such as changes in inertia or external disturbances. (Sarhadi, 2025)

Traditional attitude control technqiues are primarily designed for continuous torque commands, but in practice, the real actuators on board the spacecraft often produce discrete on-off pulses to orient the satellite. This is one of the main significant gaps in the control methods and is addressed by the inclusion of relay and deadzone blocks in conjunction with PID controllers. But these often introduce chattering, which manifests as a wastage in fuel consumption and actuator wear in real-world situation. Khosravi and Sarhadi (2016) addressed this in their research with the introduction of Pulse-Width Pulse-Frequency modulator, which converts the continuous torque signal into smoother, manageable on-off pulses and reduces the unnecessary fast thruster cycling.

Their study highlighted the need for more intelligent and adaptive control methods as traditional tuning approaches do not give best results for systems with multiple parameters with non-linear associations.

Sarhadi (2025) expands more on this limitation in his research based on the comparison between PID, Model Predictive Control (MPC) and Machine Learning (ML) based controllers across a set of standard performance and tracking criteria. The study shows that while PID based controllers remain robust and predictable for well-defined linear systems, machine learning controllers dominate in uncertain and non-linear environments as their learning policies are often derived from direct interactions with the environment and not as the result of a pre-defined mathematical model.

3.2 Reinforcement Learning for Control

Reinforcement Learning is a paradigm of Machine learning where the agent learns control laws by directly interacting from the environment and receives rewards or penalties based on its steps to reach the target. RL is preferable than other methodologies such as supervised learning as it does not require any labelled training data and the agent learns optimal behaviour through trial and error. Due to its ability to learn control policies and adaptability to dynamic and uncertain environments, RL techniques are being chosen for real-time control applications on board a satellite. (Elkins, Sood and Rumpf, 2020) Though instances of inclusion of RL-based controllers in spacecraft control systems are limited, the results obtained through preliminary studies demonstrate superior performance of RL over other traditional controllers. In a study where controllers were tested in a Mars descent and landing simulation,

the integrated guidance and control system, which used RL based framework showed an increase in landing precision and robustness to noise and uncertainity. In some of the existing literature on RL techniques, Deep Q-Network has been used to stabilize spacecraft attitude after any momentum-changing event such as payload decoupling or extreme fuel depletion. (Elkins, Sood and Rumpf, 2020) DQN network introduces two new innovations, experience replay and target networks that help stabilize the training and enabled convergence. Since it operates on discrete control spaces, selecting the action with highest estimated Q value (the expected cumulative total reward for the particular action in that state), it is suitable for problems involving on-off commands such as the spacecraft actuators. (Lillicrap et al., 2015)

For continuous action spaces, a learning algorithm called the Deep Deterministic Policy Gradient (DDPG) was propsed by Lillicrap et al. (2015), which maintained two networks known as the actor and critic network. The actor network was responsible for mapping the states to the actions deterministically while the critic network judged the quality of the state-action pair. DDPG serves as the baseline for evaluation of continuous controller in this project.

3.3 Deep Reinforcement Learning for Spacecraft Attitude Control

The application of deep reinforcement learning to the spacecraft, particularly nanosatellites has gained signifcant attention over the decades. Se Marques (2021) has investigated the use of DRL in intelligent adaptive control of satellites and has demonstrated that controllers based on DRL and neural networks learn attitude stabilization policies without the need for an explicit mathematical model. The study also demonstrated superiority of DRL based controllers over traditional methods especially in scenarios where there was parameter uncertainty or external disturbances.

In a comprehensive study conducted by Vedant et al. (2019), Proximal Policy Optimization was applied to both momentum management and momentum exchange attitude control systems. They introduced a new training approach called the curriculum training where the difficulty was increased gradually to help facilitate convergence. Inertia randomisation was applied during the training which helped the agent to control satellites across a mass range of 0.1 to 100000 kg without retraining. This approach influenced the domain randomisation method adopted in this project, where initial conditions and disturbances are randomised during DQN training to achieve convergence. Vedant et al. (2019) has also proposed a conditional reward function, with additional rewards and penalties for magnitude of positional or velocity error.

In a 2025 study, Pérez-Muñoz et al. demonstrated that Deep Reinforcement Learning (DRL) can effectively direct satellites in real-time. They discovered that DRL algorithms are efficient enough to run on the limited, low-power computers currently used in space. Furthermore, the AI worked effectively even when the environment was uncertain or the satellite's sensors were unable to view everything, demonstrating that DRL is prepared to deal with the messy realities of genuine space missions.

3.4 Research Gap and Project Motivation

The existing literature shows that DRL provides significant advantages over traditional controllers for spacecraft attitude control. However, several gaps remain. There is little research on performance of on-off thruster control using DRL and majority of the existing studies focus on continuous torque actuators and reaction wheel systems.

Additionally, while DDPG has been applied to continuous attitude control, its incompatibility with the discrete on-off actuators, which results from the mismatch between the control policy gradients and discrete action spaces, has not been thoroughly assessed and compared against DQN, which is inherently made for discrete action methods.

These gaps are addressed in this project by implemention and comparison of DDPG continuous control and DQN on-off control for single-axis satellite attitude stabilisation. The project assesses both algorithms in five test scenarios: nominal tracking, large angle recovery, inertia variation, detumbling, and disturbance rejection. The DQN framework incorporates training enhancements such as domain randomisation and disturbance augmentation, which are directly influenced by the constraints found in the DDPG baseline and the results of Vedant et al. (2019).

3.5 Summary

The following is a summary of the important literature discoveries that directly inform this project: Khosravi and Sarhadi (2016) used PSO-PID-PWPF to construct a benchmark settling time of ~3 seconds for J=11.4 kg.m² spacecraft, allowing for performance comparison. Vedant et al. (2019) demonstrated curriculum learning and inertia randomisation for robust multi-spacecraft RL control, which inspired the domain randomisation approach utilised here. Lillicrap et al. (2015) and Mnih et al. (2015) developed the core DDPG and DQN algorithms, respectively. Sa Marques (2021) and Pérez-Muñoz et al. (2025) confirmed DRL's feasibility for spacecraft attitude control applications.

4. METHODOLOGY

4.1 Satellite Dynamics Model

The satellite attitude dynamics are modelled using Euler's rotational equation of motion, for a single-axis rigid body system. With assumptions such as no cross-coupling and by neglecting any flexible modes, the angular dynamics of the spacecraft can be descibed by:

$$\frac{d^2(\theta)}{dx^2} = \frac{T}{J}$$

Equation (4.1)

Where θ is the attitude angle in radians, T is the applied control torque in Newton-metres and J is the moment of inertia of the spacecraft in kg.m^2 . This equation represents the relationship between the applied torque and the angular acceleration of a rigid body system which is rotating about a single axis. The transfer function of the spacecraft plant can be obtained by applying Laplace transformation, which yields the equation:

$$\frac{\theta(s)}{T(s)} = \frac{1}{J \cdot s^2}$$

Equation (4.2)

This double integrator model represents the simplest model for single-axis attitude dynamics matching the spacecraft plant model used by Khosravi and Sarhadi (2016). The moment of inertia is set to 11.4 Kg.m^2 , adopted from example 1 of Khosravi and Sarhadi (2016) , providing a comparable benchmark for performance evaluation. The continuous dynamics of the model is converted into discrete time steps using Euler's method, with the sample time being 0.1 seconds.

$$\theta(k+1) = \theta(k) + Ts \cdot \frac{T(k)}{J}$$

Equation (4.3)

All the four controllers have been designed and tested using the same plant model, so that the performance differences arise solely from different controller designs.

4.2 TRADITIONAL CONTROLLERS

The PID + Relay and PID + Dead zone controllers were implemented in MATLAB/Simulink using models provided by the project supervisor (Sarhadi, 2025, pers. comm.), with controller gains tuned by the author as described in Sections 4.2.1 and 4.2.2.

4.2.1 PID controller with Relay Block

The first baseline controller is a combination of traditional PID controller with a relay block that converts the continuous PID control commands into on-off control signals. The PID control law is given by the equation:

$$\tau(t) = K_p \cdot e(t) + K_i \cdot \int e(t)\ dt + K_d \cdot \frac{de(t)}{dt}$$

Equation (4.4)

Where $e(t)$ is the attitude error (the difference between the desired and current attitude) and K_p , K_i , K_d are the proportional, integral and derivative gains respectively. The relay block produces discrete thruster command of $+A$, 0 , $-A$ newton-metres from the continuous PID output. A represents the thruster force level wich is taken to be 1 N.m. The proportional, integral and derivative gains are tuned through iterative process, as each of the gain affects the controller's ability to achieve stable tracking. The controller was calibrated with a proportional gain of 20 to maximise speed while avoiding oscillations, and an integral gain of 0.12 to reduce windup, overshoot, and increased settling time. Finally, the derivative gain was adjusted to 39.7 to serve as an important "brake," smoothing the system's reaction and counteracting the destabilising effects of the other two gains. These gains have been unchanged across all the five test cases to judge the controller's robustness without retuning. The addition of relay block induces oscillations, that is, the controller oscillates around the reference in a periodic pattern resembling 'switching.' This is known as limit cycle behaviour or 'chattering' , where the controller uses a greater number of on-off pulses, switching to the extemes rapidly when it is near the reference. This extreme switching has real-world consequences such as high fuel consumption and actuator degradation over time.

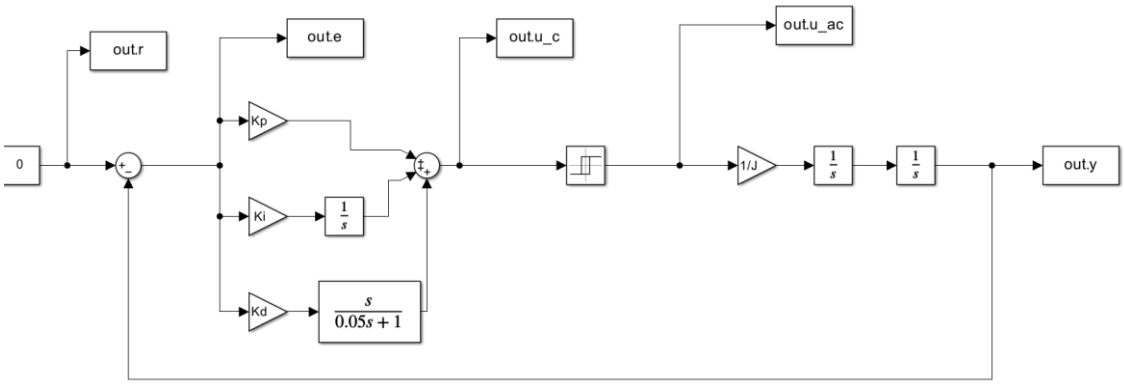


Figure 1 PID Controller model with Relay block. (adapted from Sarhadi, 2025, pers. comm.)

4.2.2 PID Controller with Relay and Deadzone Block

In the second baseline controller, there is an addition of a MATLAB function block in which both the relay and deadzone functions have been hardcoded. In control systems, dead zone refers to a specific band or range of inputs where the controller takes no action. The purpose of adding the deadzone block in this system is to prevent the system from reacting to minor errors, which suppresses switching to an extent. In this model, the deadzone has a threshold of ± 0.3 , therefore the relay cannot react to the attitude errors (PID output signals) in this range.

The relay switch-on and switch-off points are set to ± 0.5 with the output levels same as before to ensure that the relay is compatible with the deadzone threshold. The same PID gains are retained in this model to ensure fair comparison with the earlier model. The key difference between the first and the current model is the reduction in switching frequency due to addition of deadzone, which directly

influences the amount of thruster firing and fuel consumption is real spacecraft applications. This is directly comparable to the PWPF modulator's quasi-linear operation concept as explained by Khosravi & Sarhadi (2016).

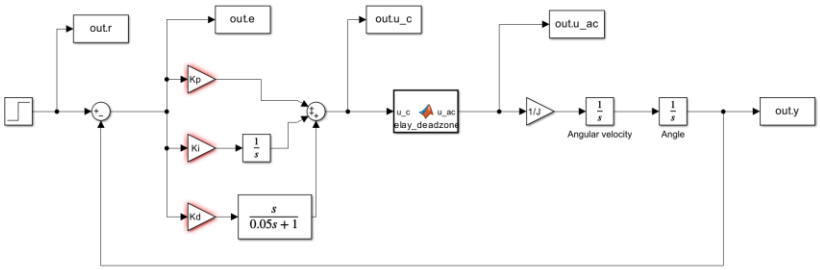


Figure 2 PID Controller Simulink Model with Relay and Dead zone (adapted from Sarhadi, 2025, pers. comm.)

4.3 DDPG Continuous Controller

4.3.1 Algorithm Overview

The Deep Deterministic Policy Gradient algorithm is an off-policy reinforcement learning method particularly designed for continuous action spaces. This algorithm contains two main networks:

- Actor network $\pi(s)$: This network acts a function approximator (typically neural network) and outputs a single value (action) for a state, rather than a probability distribution which is common in other AI algorithms.

$$a = \pi(s)$$

Equation (4.5)

  Where s is the state and a is the action.

- Critic Network $Q(s, a)$: This network evaluates the state-action pairs the actor network produces and learns to estimate the Q - value which is the total expected reward the agent will receive for a particular action (a) in that specific state (s). This essentially tells how good the action is in a particular state. The critic network estimates the Q-values for all the possible state-action pairs and chooses the pair which gives the highest Q-value.

The actor network is trained to move its output a in the direction the critic deems to be beneficial in increasing Q-value. To adapt the agent to a problem where the target is not fixed, target networks are introduced. These networks are delayed, ‘frozen’ copies of the main networks and serve as stable targets so that the network doesn’t diverge by learning from its changing predictions. DDPG is selected as the continuous controller baseline so that the DRL attitude control can be evaluated under ideal actuator conditions before applying discrete on-off control.

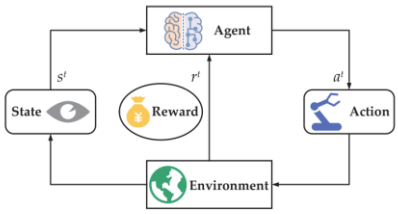


Figure 3 Schematic Diagram of RL operation process. Source: Qiu et al. (2023)

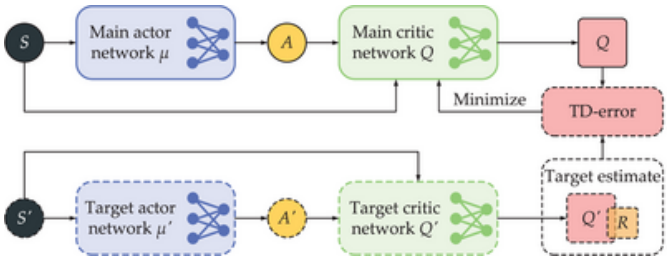


Figure 4 Schematic Diagram of the DDPG Architecture. Source: Qiu et al. (2023)

4.3.2 Observation and Action Spaces

The observation space vector consists of:

$$s = [e, \dot{\theta}]^T$$

Equation (4.6)

Where e is the attitude error ($\theta_{ref} - \theta$) in radians and $\dot{\theta}$ is the angular velocity. In nominal tracking cases, these initial conditions are set zero. The observation bounds defined for the error and the angular velocity are defined as $e \in [-2\pi, 2\pi]$ rad to ensure the satellite doesn't complete a full rotation and $\dot{\theta} \in [-5,5]$ rad/s, these are consistent with physical realistic conditions. This torque range is dieliberatley assumed wider compared to that of PID baseline (± 1 N.m) to reflect the ideal continuous actuator conditions assumed in the DDPG controller system.

4.3.3 Actor and Critic Network Architecture

The actor network consists of a fixed architecture, with just the number of layers and non-linear activation functions differing according to the purpose of the agent.

- The first layer is always the 'Feature Input Layer', as it serves as a doorway to the next layer, and its only purpose is to simply pass on the two-dimensional observation values to the next layer.
- The feature input layer is followed by fully connected hidden layers of neurons with non-linear activation functions in between the layers. The number of layers and neurons affect the agent's capacity to learn complex patterns. Most of the RL studies start with 64 neurons as an average as too many neurons can increase the training time and too less neurons may lead to failure in complex tasks.
- The non-linear activation function helps the agent to adapt to non-linear dynamics, without it, the actor network just might remain as a simple linear regression model and might not adapt to real-world complex data. The activation function employed here is the Rectified Linear Unit (ReLu), which acts like an on-off switch. Its action is based on the function:

$$f(x) = \max(0, x)$$

Equation (4.7)

If the input from the neuron is negative, it becomes inactive in the next layer after passing through ReLu layer. This way, only relevant neurons stay active in further layers, contributing to pattern formation and recognition. Another non-linear activation function called the hyperbolic tangent (tanh) layer reduces the value to fit them into the range of –1 to 1, to ensure they remain bounded.

- The value obtained after application of tanh activation function is passed through scaling layer, where it is mapped to the physical torque range of ± 5 Nm.

The critic employs a branching architecture, with one 64-neuron FC layer for the action input and three for the state. Their outputs are combined using an addition layer and ReLU activation, culminating in a single linear neuron, giving the estimated Q-value.

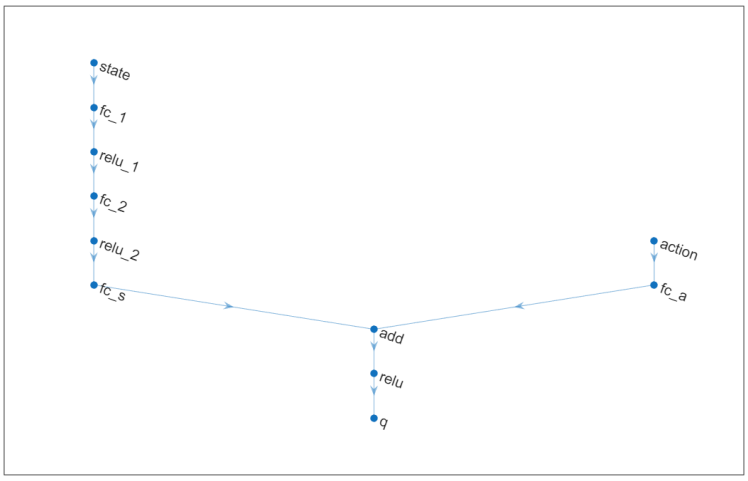


Figure 5 Critic network architecture. Source: Author

4.3.4 Reward Function

The reward function used in this study is adapted from the conditional reward framework proposed by Vedant et al. (2019) . It has been modified for single – axis scalar attitude control. The total reward at each timestep is given by:

$$r = base_r + c$$

Equation (4.8)

Where base reward is the nominal reward condition that penalises the attitude error, angular rate and control effort while c is the conditional reward term.

$$base_r = -(error^2 + 0.1 \cdot angular\,rate^2 + 0.0001 \cdot \tau^2)$$

Equation (4.9)

Reward / penalty	Condition for the reward	Note
$c = +500$	$if\ \lvert e \rvert \leq 0.1\ AND\ \lvert \dot{\theta} \rvert \leq 0.1$	Goal achieved
$c = +50$	$if\ \lvert e \rvert \leq 0.1$	Attitude achieved
$c = -50$	$if\ \lvert e \rvert \geq 1.0\ OR\ \lvert \dot{\theta} \rvert \geq 1.0$	Catastrophic failure

$c = -5$	$if\ \lvert e \rvert \geq 0.5\ OR\ \lvert \dot{\theta} \rvert \geq 0.5$	Moderate deviation
$c = 0$	otherwise	

The large positive bonus and the penalties are designed to give the agent a clear learning path so that it can use both exploration and exploit its knowledge to find the target state. The energy penalty term in the base reward ensures the agent stops applying torque once the satellite has settled successfully.

4.3.5 Training Configuration

The hyperparameters used in the training are as follows:

Hyperparamter	***Value***	***Justification***
Discount factor	0.99	Appropriate for attitude control as the goal requires long term-planning , making the agent priortize future rewards
Target Smooth factor	0.001	Standard value from Lillicrap et al. (2015), prevents instability caused by rapidly moving target values
Buffer length	10^6	Reduces correlation between samples, stabilises learning
Mini-batch size	128	Larger than default 64 to provide more stable gradients
Actor learning rate	$1*10^{-4}$	Standard practic in actor-critic literature , slower than critic learning rate to allow for the critic to evaluate the actions
Critic learning rate	$1*10^{-3}$	10 times faster than actor learning rate as critic must learn to estimate Q-values quickly and accurately
Noise variance	0.5	High initial noise variance allows the agent to explore the action broadly in early training episodes
Noise decay rate	10^{-3}	Faster rate selected as it was observed in early training runs that agent was exploring and acting randomly even though it had found the right actions causing divergence
Max episodes	1000	Computational feasibility
Steps per episode	500	To match the 50s simulation window present in other models
Stop threshold	400	With c = +500 success bonus and typical base_r = -100, the expected reward would be 400, the maximum reward the agent could get in successful episodes

Table 1: Training hyperparameters

4.4 DQN On-Off Controller

4.4.1 Algorithm Overview

The Deep Q-Network (DQN) algorithm, proposed by Mnih et al. (2015), is an off-policy reinforcement learning method designed by combining reinforcement learning with a class of artifical neural networks called the deep neural networks. This algorithm method takes discrete values as its action

spaces therefore it is compatible with the on-off thruster actuation commands required in real spacecraft systems. Like DDPG, DQN estimates the Q-function using a deep neural network and chooses the discrete action with highest estimated Q-value.

4.4.2 Observation and Action Space

The observation space employed in DQN algorithm is same as that of DDPG controller, containing attitude error and angular rate :

$$s = [e, \dot{\theta}]^T$$

Equation (4.10)

The action space is discrete, comprising of three possible thruster commands:

$$a \in \{-1, 0, 1\}Nm$$

Equation (4.11)

Where each of the value represents negative thruster firing , thruster off and positive thruster firing (opposite direction to negative thruster firing in a literal sense) respectively. The thruster force here (A = 1 Nm) matches the force used in the PID baseline controllers, ensuring fair and direct performance comparison.

4.4.3 Network Architecture

A fully connected feedforward structure is used by the DQN architecture to translate three discrete action Q-values to a 2D observation space. It consists of three 64-unit hidden layers with ReLU activations, followed by a three-neuron linear output layer that simultaneously estimates Q-values without the need for an activation function. The action that is selected at each time step is:

$$a^* = \arg\max[Q(s, -1), Q(s, 0), Q(s, +1)]$$

Equation (4.12)

The above vector Q-value formulation is performed directly by MATLAB's rlVectorQValueFunction , and it produces all three Q-values (for the three discrete action spaces) simultaneously.

4.4.4 Training improvements

Two training improvements have been implemented into the DQN framework to counteract some of the limitations observed in DDPG baseline performance.

- Domain randomisation: The nature of initial conditions was changed from fixed to random, that is, the initial attitude angle is sampled uniformly from $\theta_0 \in [-2\pi, 2\pi]$ rad using ;rand; function in MATLAB, along with randomisation of initial angular rate from this range $\dot{\theta}_0 \in [-5, 5]$ rad/seconds. This approach was adopted from Vedant et al. (2019) 's study to this project as the agent is exposed to a wider range of starting conditions, improving its robustness and recovery.
- Disturbance inclusion: Each training episode begins with a uniformly sampled random constant disturbance torque d ∈ [-0.3, +0.3] Nm. The steady state error limitation seen in the

DDPG controller in Case 3 is addressed by encouraging the agent to develop intrinsic disturbance rejection capability.

4.4.5 Reward Function

The DQN reward function has the same structure as the DDPG reward function, including threshold values and conditional bonuses and penalties. Therefore, any performance discrepancies are assured to be due to the algorithm and action space design rather than the differences between reward functions.

4.4.6 Training Configuration

Unlike DDPG, which uses additive noise for exploration, DQN agent is trained using epsilon-greedy exploration strategy. With initial epsilon $\varepsilon = 1$ value, the agent chooses an action randomly from the available action space. This ensures that the agent is exploring all the paths to reach the target rather than being stuck in local optimum. The epsilon decay rate is 10^{-3} , which is faster decay rate to ensure the agent is not tuck just exploring but exploiting its knowledge to reach the target while gaining maximum rewards. All the other hyperparameters, including the stopping criteria, match those of DDPG controller so that the training efficiency can be evaluated and compared fairly.

4.5 Test Case Definitions

Five test scenarios have been defined to evaluate and compare all four controllers. These test cases represent realistic operating conditions of spacecraft missions.

- **Case 1 – Nominal Step Response:** In this scenario, the satellite is commanded to reach a target attitude of $\theta_{ref} = 1\,rad$ from rest at $\theta_0 = 0\,rad$. This case serves as the baseline tracking scenario and nominal performance metrics such as settling time, overshoot percentage are established.
- **Case 2 – Large Initial Angle Recovery**: The satellite's initial angular position (θ_0) is set to a large angle such as 5 radians with zero initial angular velocity and must recover to θ_{ref} = 1 radian. This evaluates the controller's ability to handle large attitude erros which reflects the realistic scenario of satellite post-separation from the launch vehicle.
- **Case 3 – External Disturbance Rejection:** A constant torque of d = 0.5Nm is designated and applied as an external disturbance torque throughout the simulation, with the satellite retaining the same initial conditions as in case 1. This case tests the controller's robustness to constant external forces which, in real spacecraft missions, represent the atmospheric drag or the solar radiation pressure.
- **Case 4 – Inertia Variation:** The moment of inertia is changed from 11.4 kg.m^2 to 1000 kg.m^2 which is approximately 88 times the baseline training value. The other controller parameters are changed back to the nominal values. This evaluates how efficient is the controller to parameter uncertainity without retuning.

- **Case 5 – Detumbling:** This represents the detumbling scenario of the satellite after launch sepaation where both attitude and angular rate should be brought to zero. In this test case, the satellite begins with a large initial angle of $\theta_0 = 5\,rad$ and an angular rate of $\dot{\theta}_0 = 2$ radians/second. The controller's mission is to bring the satellite to a reference position of 0 rad.

4.6 Performance Metrics

All the controllers are evaluated against this set of standardised performance metrics:

Performance metrics	
Settling Time	Settling Time is the first time, expressed in seconds, that the attitude angle enters and stays within a ±2% band of the reference value. Since the reference is zero in the detumbling case, a fixed band of ±0.05 rad is employed.
Percentage overshoot	Maximum deviation of the attitude angle above the reference value. For recovery cases, this metric is not valid.
Steady State Error	Defined as the mean absolute error over the final 50 simulation steps, including any offset from the reference after settling.
Peak Torque	Defined as the maximum absolute torque applied during the simulation. It representrs the maximum actuator demand.

Table 2: Standardised Performance Metrics

5. Results and Discussion

This chapter presents the simulation resulta for all the four controllers across the fuce test cases mentioned in Methodology. Section 5.1 covers the performance results of the PID-based controllers, Section 5.2 presents the results of the DDPG continuous controller, Section 5.3 presents the DQN on-off controller performance and Section 4.4 provides insights into the cross-controller comparison.

5.1 Traditional Controllers (PID Baseline)

5.1.1 PID with Relay

In this controller model, a relay block is employed to convert the continuous PID output into a binary ± 1 N.m torque command which represents the on-off thruster signals. The controller gains were set as $K_p = 20$, $K_i = 0.12$ and $K_d = 39.7$ for a spacecraft of inertia of J = 11.4 kg.m^2 .

Case 1 – Nominal Step Response

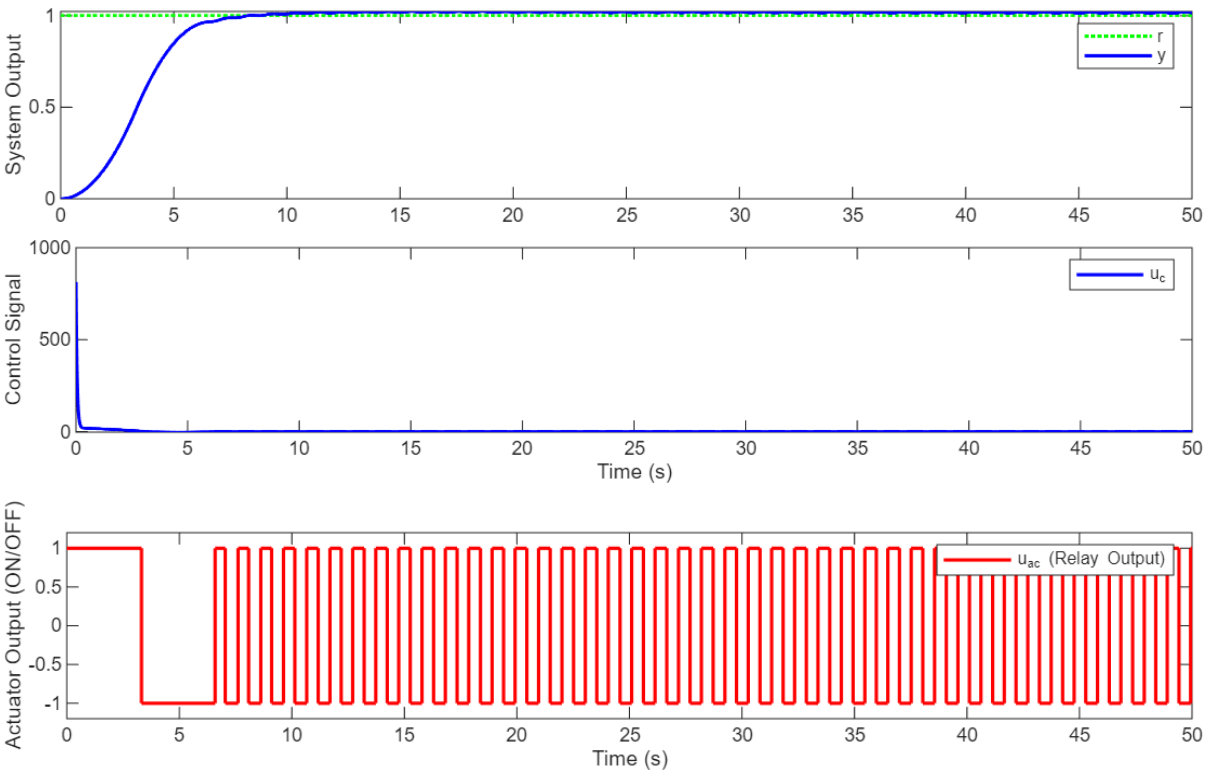


Figure 6 PID with relay nominal step response plots

In this case, the controller's task was to bring the sateliite to an attitude of $\theta_{ref} = 1\,rad$ from rest at $\theta_0 = 0\,rad$. The controller has achieved a stable response, settling within approximately 10-11 seconds. The actuator output plot showed that there were wide torque pulses during the initial approach (as the satellite was in path towards the reference point) which narrowed progressively as the satellite neared its setpoint. There was low-frequency limit cycle behaviour at the end as the

controller was using its thrusters continuously to remain at the desired attitude. No steady state error was observed as the integral term successfully eliminated any residual offset.

Case 2 – Large Initial Angle Recovery

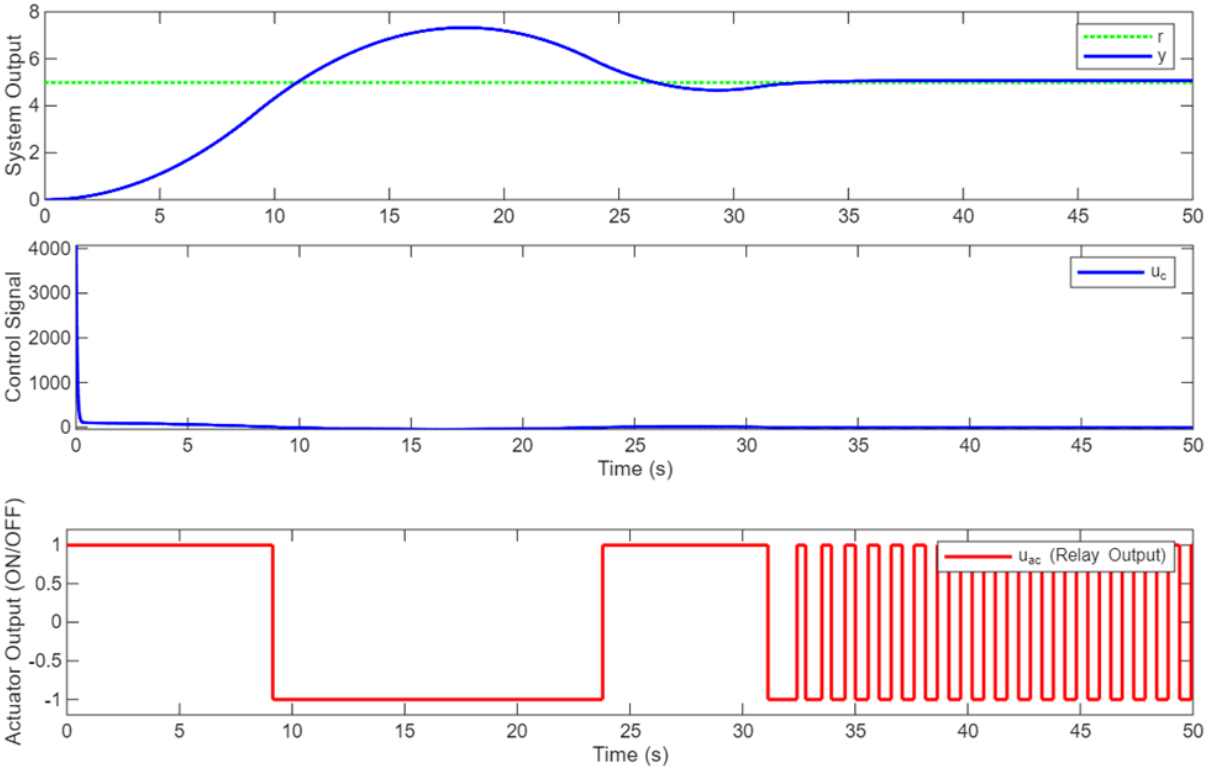


Figure 7 PID with Relay case 2 response plots.

This test case utilised a reference attitude of $\theta_{ref} = 5$ rad and zero beginning circumstances. Because the PID controller only responds to tracking errors $(e = \theta_{ref} - \theta_0)$, this is functionally equivalent to starting at $\theta_0 = 5\,rad$ with a zero reference. Using nominal gains, the controller produced a +1.23 rad overshoot and settled in 34–36 seconds. The larger manoeuvre increased the magnitude of the control signal as well as the settling time, indicating that more effort was necessary to compensate for the increased tracking error. During the initial approach, the relay remained saturated, evident from the wide thruster pulses, and the width decreased progressively as the satellite reached the setpoint. The integral gain was sufficiently effective to eliminate residual offset even in large manoeuvre cases.

Case 3 – External Disturbance Rejection

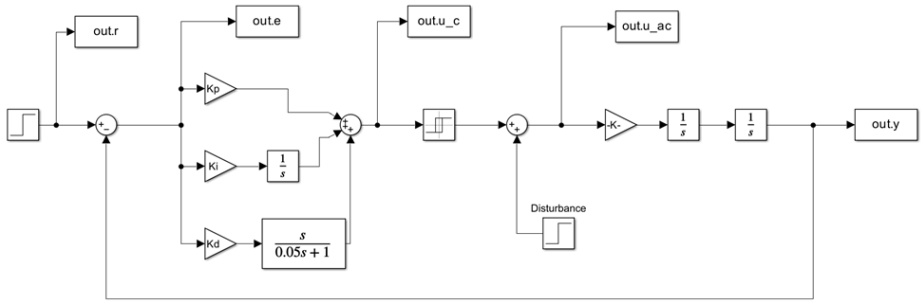


Figure 8 External disturbance injection (adapted from Sarhadi, 2025, pers. comm.)

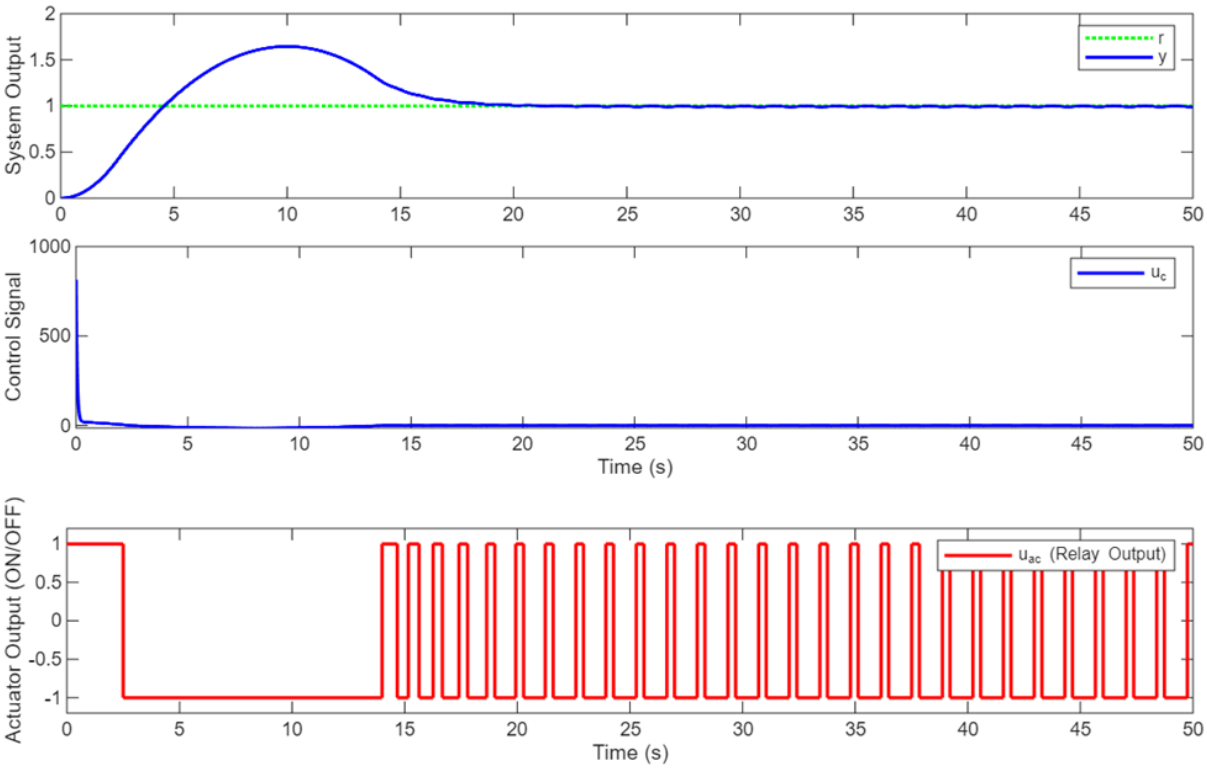


Figure 9 PID with relay Case 3 response plots (Generated by MATLAB)

The application of constant external disturbance torque ($d = 0.5\,N \cdot m$) was simulated by addition of a step block and a sum block in the Simulink model. The controller successfully tracked the 1 rad reference even under constant disturbance demonstrating that effectiveness of the integral term in rejecting the steady state disturbance. On comparing to Case 1, it can be observed that there is an signifcant increase in both percentage overshoot and the switching frequency of the relay. The relay rapidly switched between $\pm 1\,N \cdot m$ pulses to counteract the persistent disturbance, as shown by the appearance of thin, rapid pulses in the actuator output plot. In real spacecraft applications, this behaviour directly translates to greater fuel consumption due to increased frequency of thruster firings. This is a crucial disadvantage of the relay-only PID contoller under this test case.

Case 4 – Inertia Variation

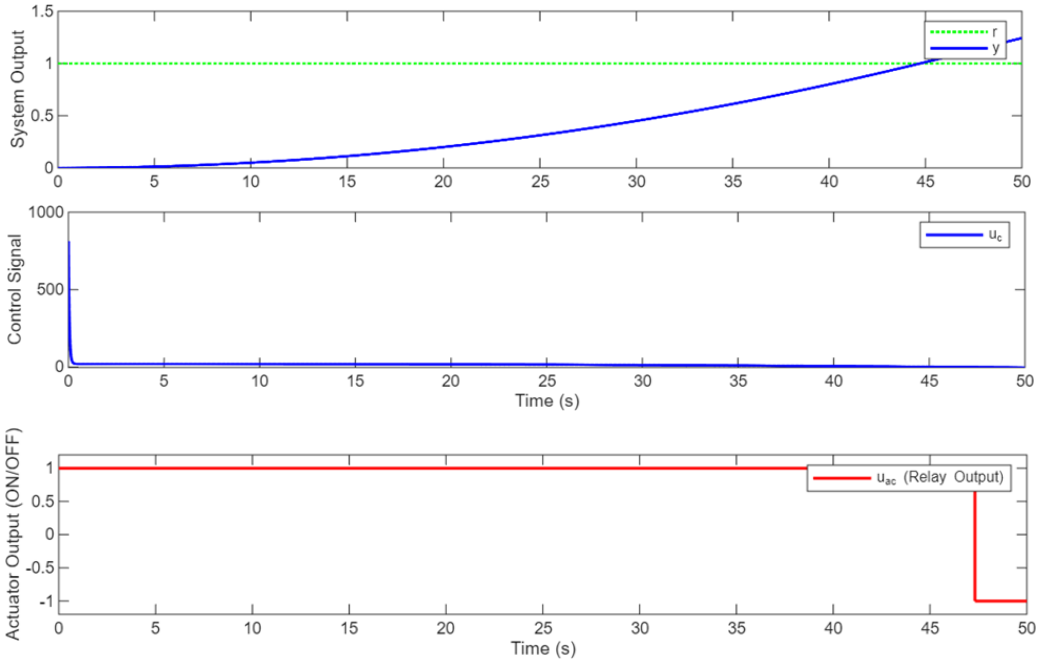
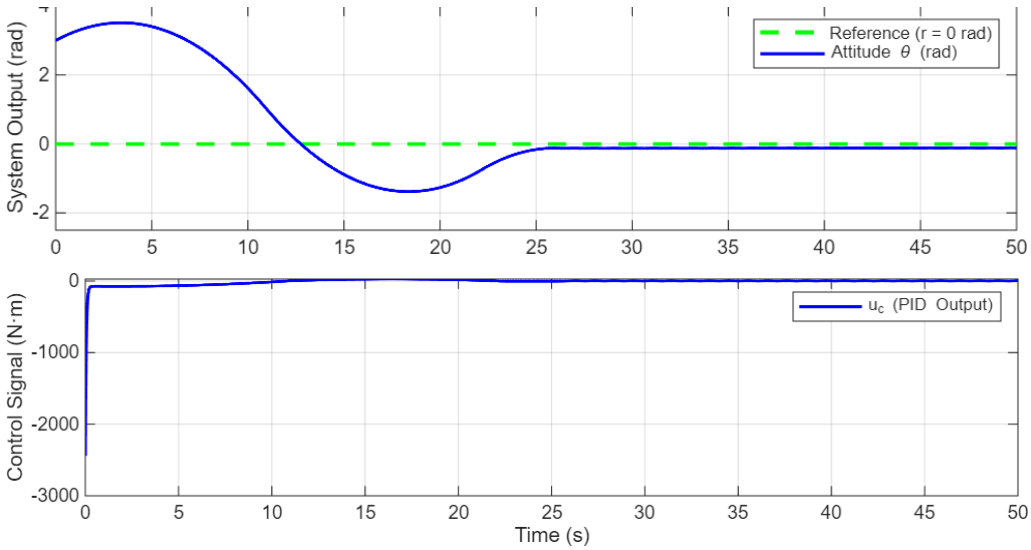


Figure 10 PID with Relay Case 4 Response Plots

When the moment of inertia was increased to J = 1000 kg·m², the controller with nominal gains lost its ability to maintain attitude. After around 48 seconds, the control effort dropped to nearly nothing, and the satellite steadily drifted away from the setpoint. The larger plant did not achieve sufficient angular acceleration with the same ±1 N·m torque as at J = 11.4 kg·m². Manual retuning restored acceptable performance, resulting in Kp = 70, Ki = 0.5, and Kd = 90. This demonstrates that the PID with Relay controller can accept considerable inertia changes, but only through manual re-tuning of the gains.

Case 5 – Detumbling



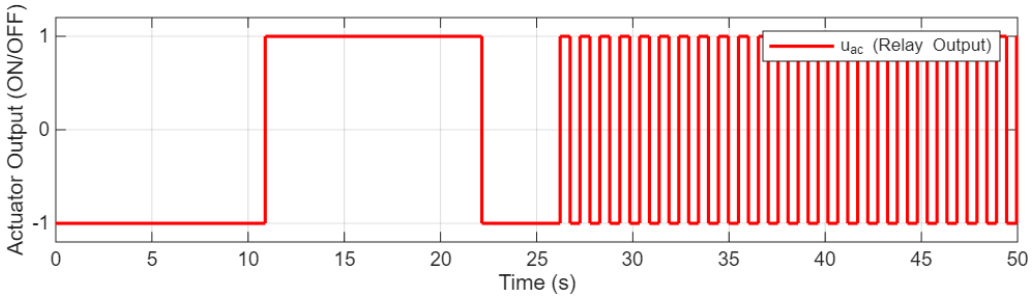


Figure 11 PID with Relay Case 5 response plots

Tracking Performance: After an initial overshoot of -1.5 rad, the system effectively neutralises the first 3 rad tumble and settles at the 0 rad reference in around 26 seconds.

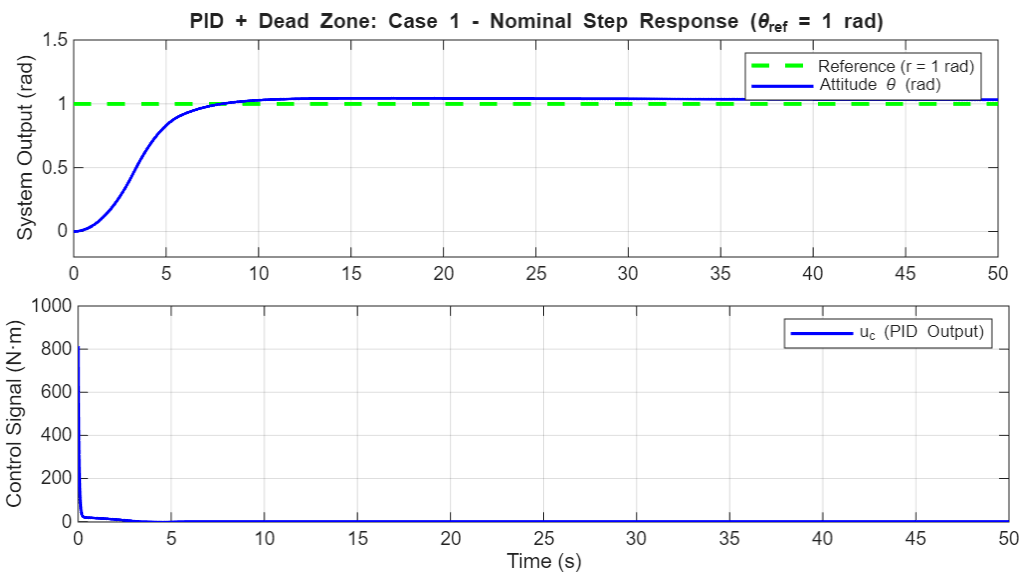
Control Action: To counteract the reverse momentum, the actuator applies the maximum braking torque (-1) for 11 seconds before switching to +1.

Steady-State Flaw (Chattering): The actuator is unable to shut down at the reference point in the absence of a dead zone. Rather, it enters a high-frequency, continuous flipping cycle between -1 and +1, which is extremely harmful to the hardware of the spacecraft.

5.1.2 PID with Relay and Dead Zone

The PID-RDZ controller enhances the fundamental design by incorporating a dead zone (± 0.3 threshold) after the relay to avoid unwanted activations near the setpoint. The relay switch points (± 0.5) must exceed the dead zone. Initial testing with thresholds near machine epsilon ($\varepsilon \approx 2.2 \times 10^{-16}$) resulted in the dead zone blocking the relay and producing uncorrected drift. Standard PID gains ($K_p = 20$, $K_i = 0.12$, $K_d = 39.7$) were kept for a fair comparison.

Case 1 – Nominal Step Response



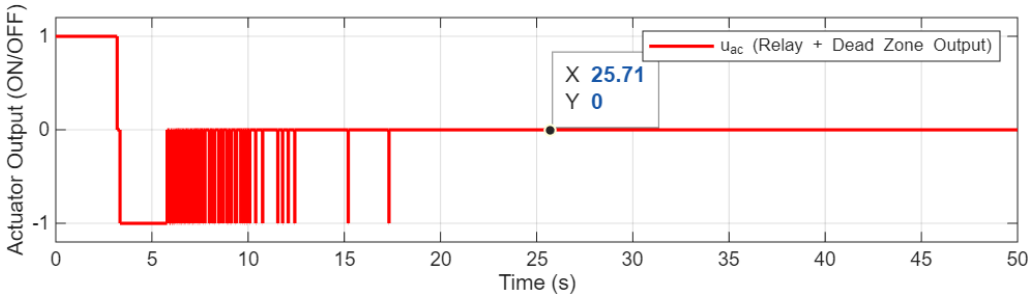


Figure 12 PID-RDZ Case 1 Response Plots

Figure 12(a) illustrates the nominal step response of the PID-RDZ controller while monitoring a target attitude of 1 radian. Initially, the system responds smoothly, accelerating the satellite with the highest positive torque. At approximately 3 seconds, it shifts to maximum negative torque to brake and avoid major overshoot. In around ten seconds, the system achieves the 1 rad setpoint.The fundamental advantage of the dead zone is proven by the zero-output condition, that is, it effectively eliminates the high-frequency steady-state chattering found in conventional relay controllers. Suppressing this continual switching is critical for realistic spacecraft missions because it reduces unnecessary fuel consumption and reduces mechanical wear on actuators, both of which extend the mission's operating lifespan.

Case 2 – Large initial angle recovery

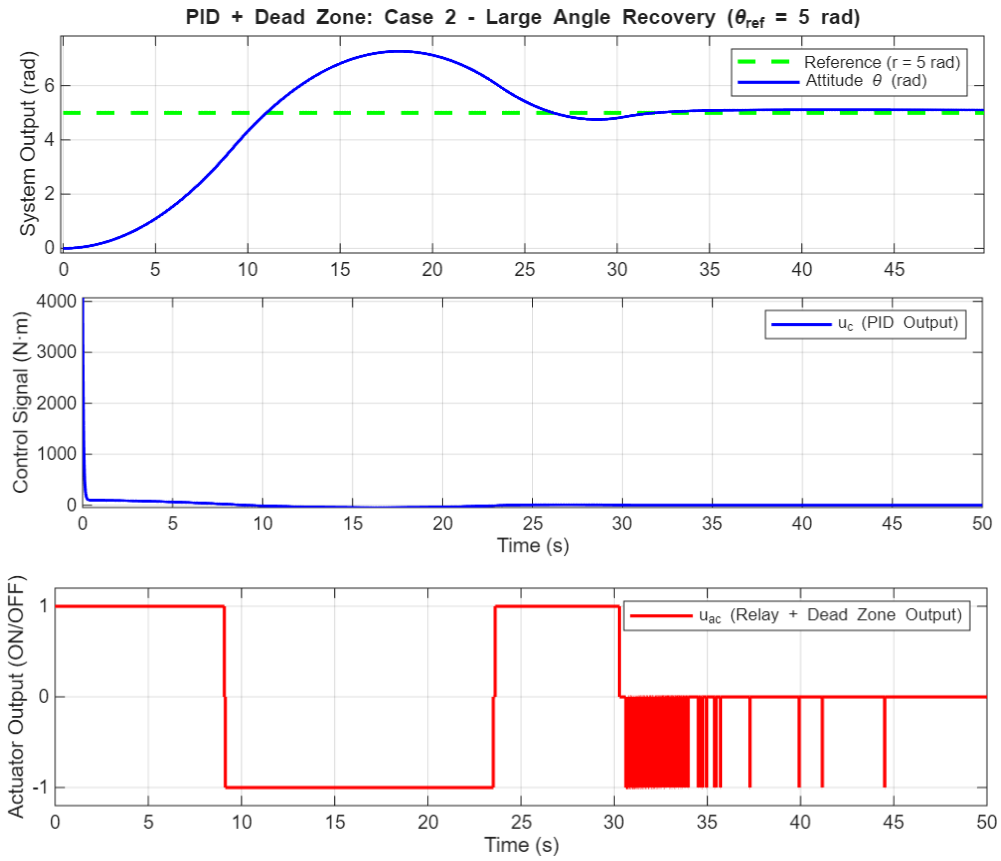


Figure 13 PID-RDZ Case 2 Response Plots

The PID-RDZ controller effectively moves the system from an initial rest state to the target in the big angle recovery case (5 radians). To close the enormous angular gap, the actuator applies a sustained positive torque (+1 N.m) for the first nine seconds. Then, it applies a sustained negative torque (-

1N.m) to arrest the momentum. An overshoot to around 7 rad is caused by this forceful initial manoeuvre, but it is promptly corrected. The system enters the dead zone by t = 30 seconds. The system reaches full steady-state stabilisation with zero steady-state error at t = 45 seconds after a short interval of sparse corrective pulses.

Case 3 – External Torque Rejection

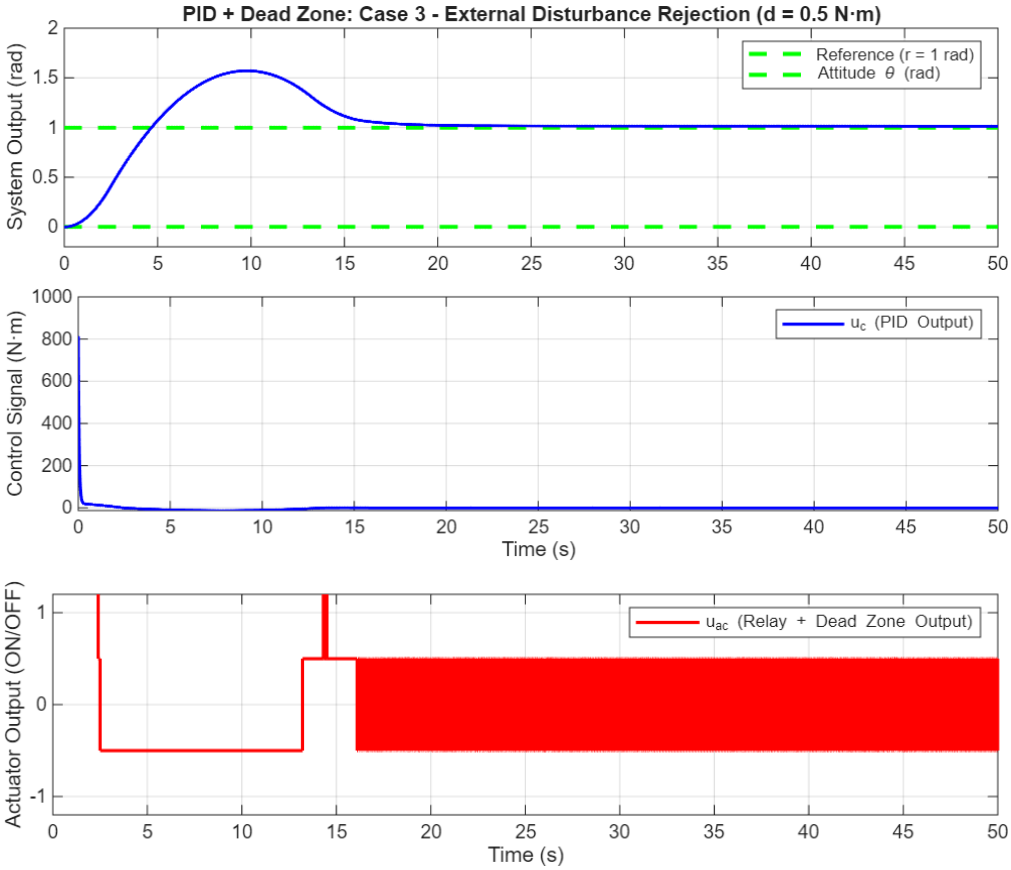


Figure 14 PID-RDZ Case 3 Response Plots

The PID-RDZ controller was tested to reject a 0.5 N·m external disturbance while tracking a 1 rad reference. The system output achieves the aim, with an initial overshoot to around 1.5 rad at 10 seconds before settling precisely at the 1 rad setpoint around the 20-second mark with 0% steady-state error. After 16 seconds, the actuator switches to continuous, high-frequency switching (chattering) between -0.5 and +0.5 to continue counteracting the persistent disturbance and maintaining the attitude. This persistent steady-state chattering provides the necessary average control effort to eliminate the extenal disturbance.

Case 4: Inertia Variation

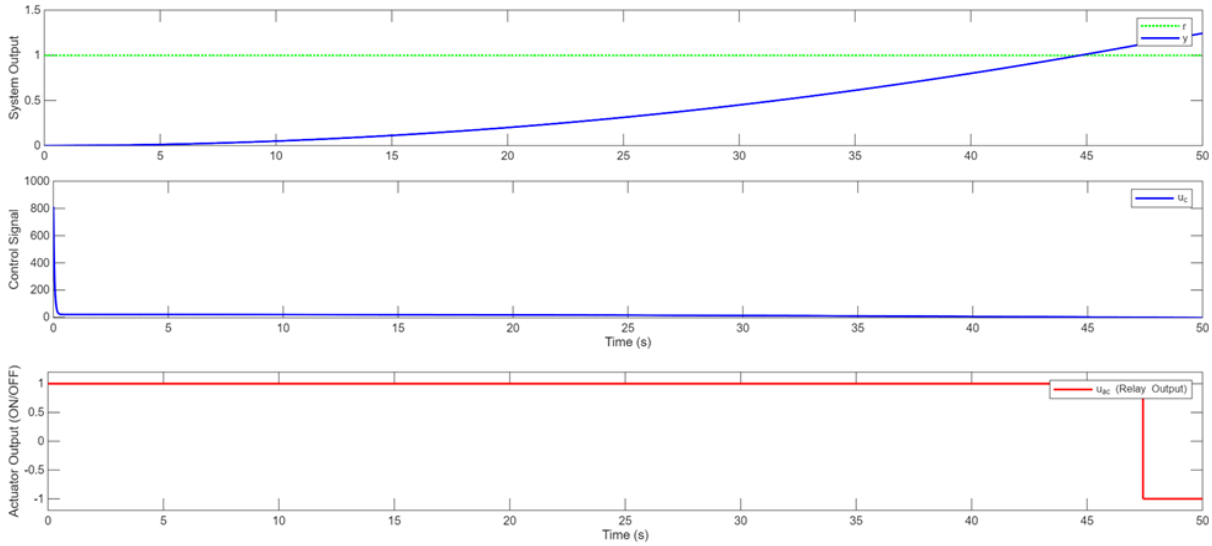


Figure 15 PID-RDZ Case 4 response plots

Under nominal gains at J = 1000 kg·m, the PID-RDZ failed similarly to the PID-R and required gain retuning for recovery. Because this limitation is inherent in the fixed-gain PID architecture rather than the actuation logic, the dead zone did not increase resistance to inertia variation.

Case 5: Detumbling

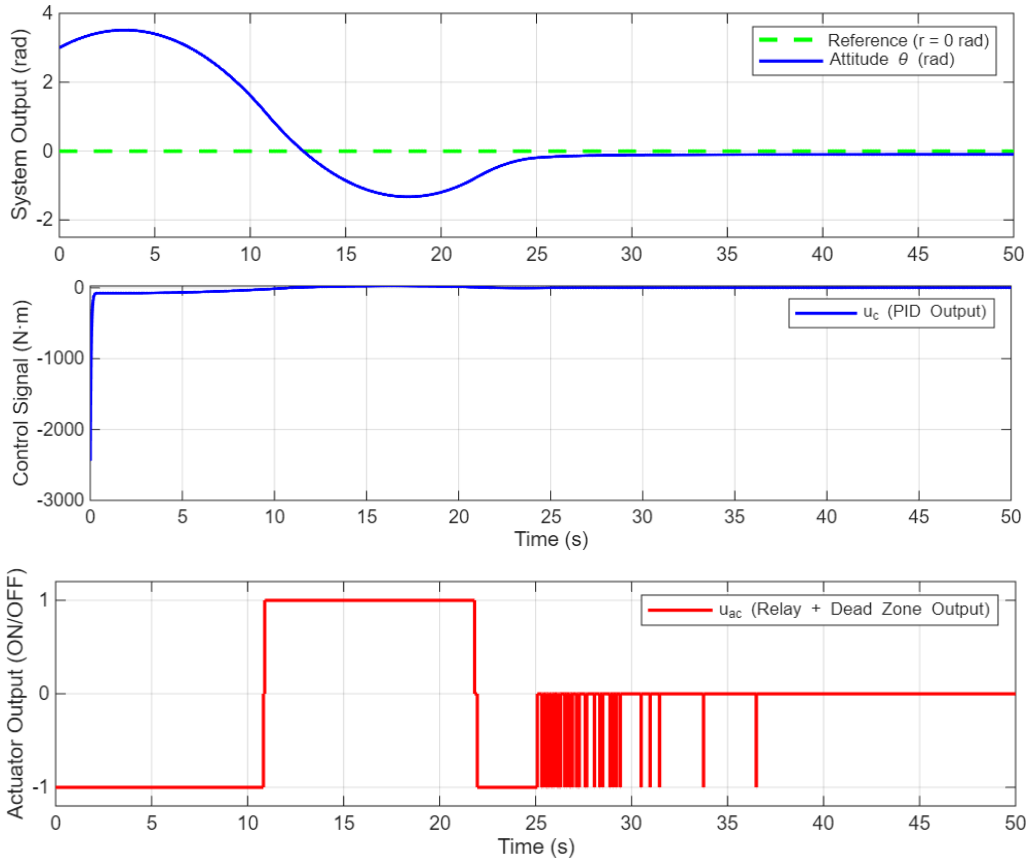


Figure 16 PID-RDZ Case 5 Response Plots

A MATLAB function block was used apply the combined effects of relay and deadzone. Figure X shows the PID-RDZ controller's performance during the detumbling scenario, which begins with an initial

attitude of 3 radians. The controller effectively stops the initial angular momentum by supplying a constant negative torque (-1 N.m) for the first 11 seconds. As the satellite approaches the zero reference, the actuator shifts to a +1 N.m state to offset the consequent overshoot, which peaks at around -1.5 rad. Within 25 seconds, the system has successfully stabilised near the intended attitude. Most significantly, the actuator output indicates the successful working of the dead zone logic; after a brief period of corrective pulsing, the control effort completely converts to a zero-output state after 36 seconds.

5.2 DDPG Continuous Controller

Case 1 – Nominal Step response

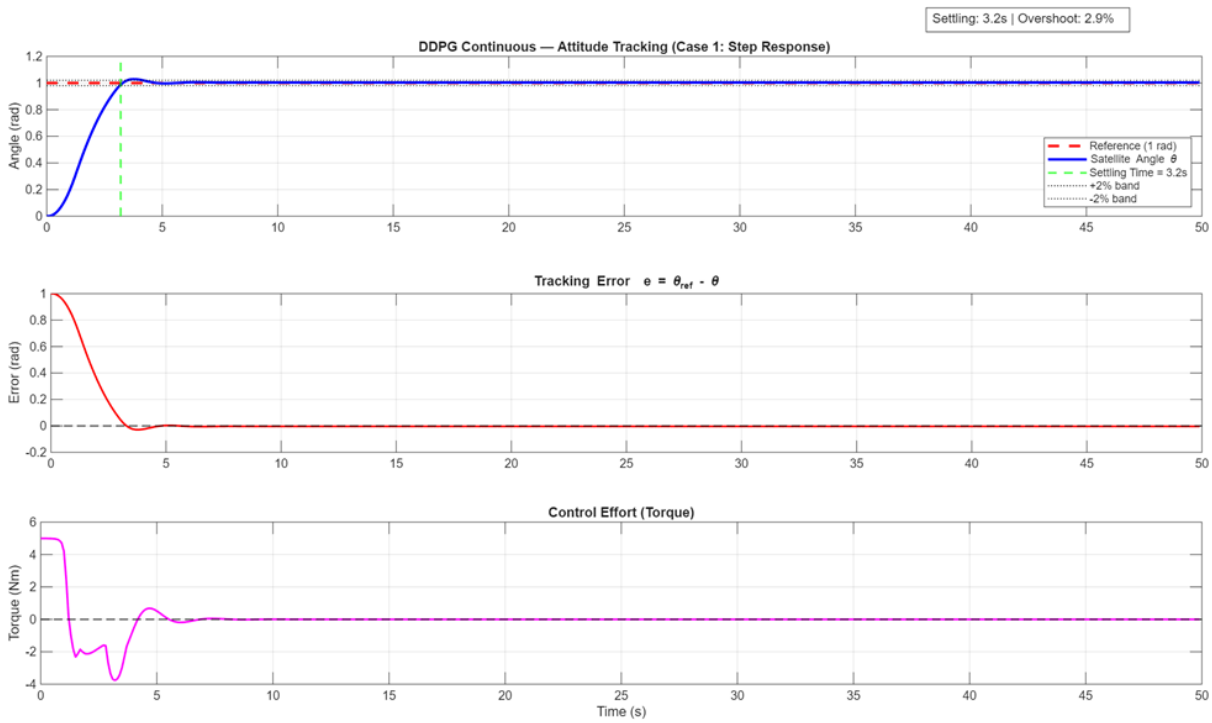


Figure 17 DDPG Controller - Case 1 Plots

The first part of Figure 17 shows the continuous DDPG controller's step reaction to tracking 1 rad reference attitude from a resting condition. The system meets the target efficiently, with a settling time of 3.2 seconds (within a 2% tolerance) and a peak overrun of 2.9%. Once stabilised, the steady-state error approaches and remains at zero. The accompanying control signal shows a smooth torque curve that peaks about 5 Nm during the initial movement and gradually decreases to virtually zero as the satellite settles. Unlike previously proven PID relay controllers, this continuous DDPG technique provides no actuator chattering, hence eliminating mechanical switching at steady state.

The DDPG agent achieves competitive nominal performance without explicit parameter tuning, as evidenced by a settling time of 3.2 seconds.

- Settling time: 3.20 seconds
- Overshoot: 2.89%
- Steady State Error: 0.0037 rad
- Peak Torque: 4.9940 Nm

Case 2 – Large Initial Angle Recovery

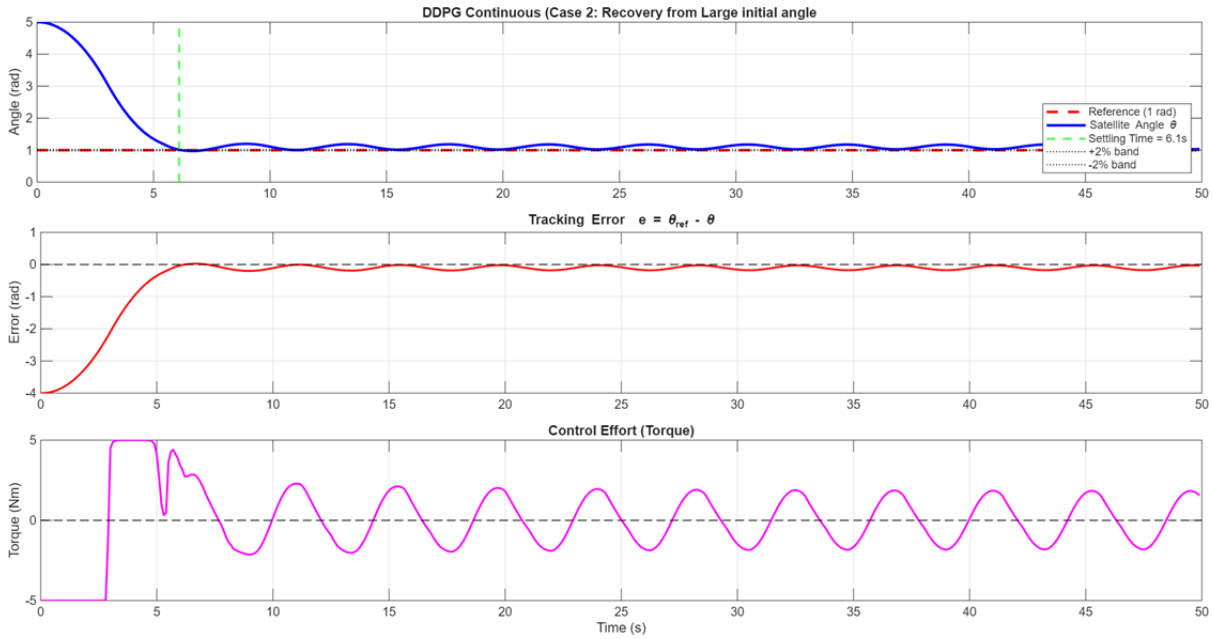
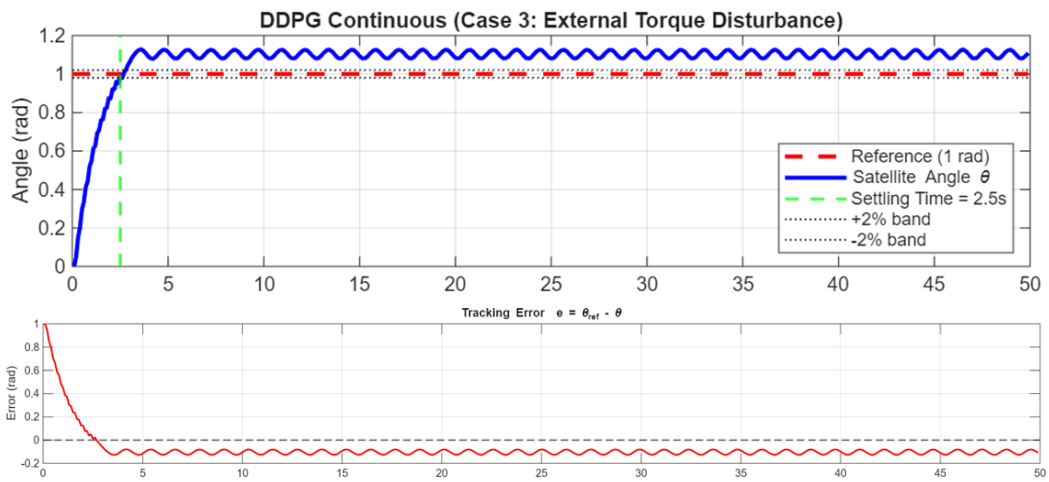


Figure 18 DDPG Controller - Case 2 Plots

The first part of Figure 18 shows the DDPG controller performing a large angle recovery from a starting attitude of $\theta_0 = 5$ rad to a 1 rad target. The controller responds quickly, employing a time-optimal switching strategy—applying near-maximum alternating torque to stop the satellite's momentum—and achieving an initial settling time of about 6.1 seconds. However, the system does not fully stabilise. For the remaining 50 seconds of the simulation, the attitude oscillates in a ±0.1 rad limit cycle around the setpoint. This steady-state instability emerges because the 5 rad beginning position falls significantly outside the agent's original training distribution, which was limited to initial conditions near $\theta_0 = 0$ rad.

- Settling time (2%) : 6.10 seconds
- Steady state error: 0.0906 rad
- Peak Torque: 5.0000 Nm

Case 3- External Torque Disturbance



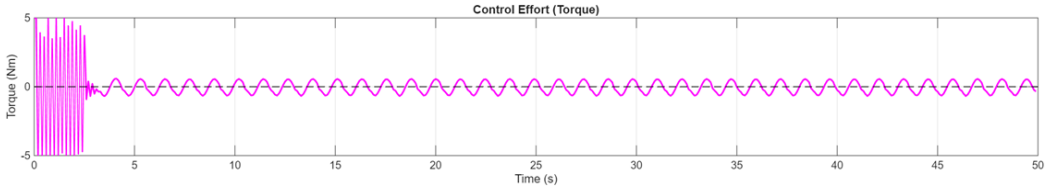


Figure 19 DDPG Controller Case 3 Plots

Under a constant 0.5 Nm external disturbance, the DDPG controller settles in 2.5 seconds with a steady-state inaccuracy of about 0.1 rad. To counteract the imposed torque, the control signal switches continuously at high frequencies. This persistent error highlights a fundamental disadvantage of the trained DDPG agent over traditional PID controllers: because the agent only processes instantaneous state data (current error and angular rate), it lacks the integral action required to eliminate steady-state offsets caused by constant disturbances and the training of the agent does not include disturbance rejection conditions.

- Settling time: 2.50 seconds
- Overshoot: 12.75%
- Steady State error : 0.1034 rad
- Peak Torque : 5.00 Nm

Case 4 – Inertia Variation

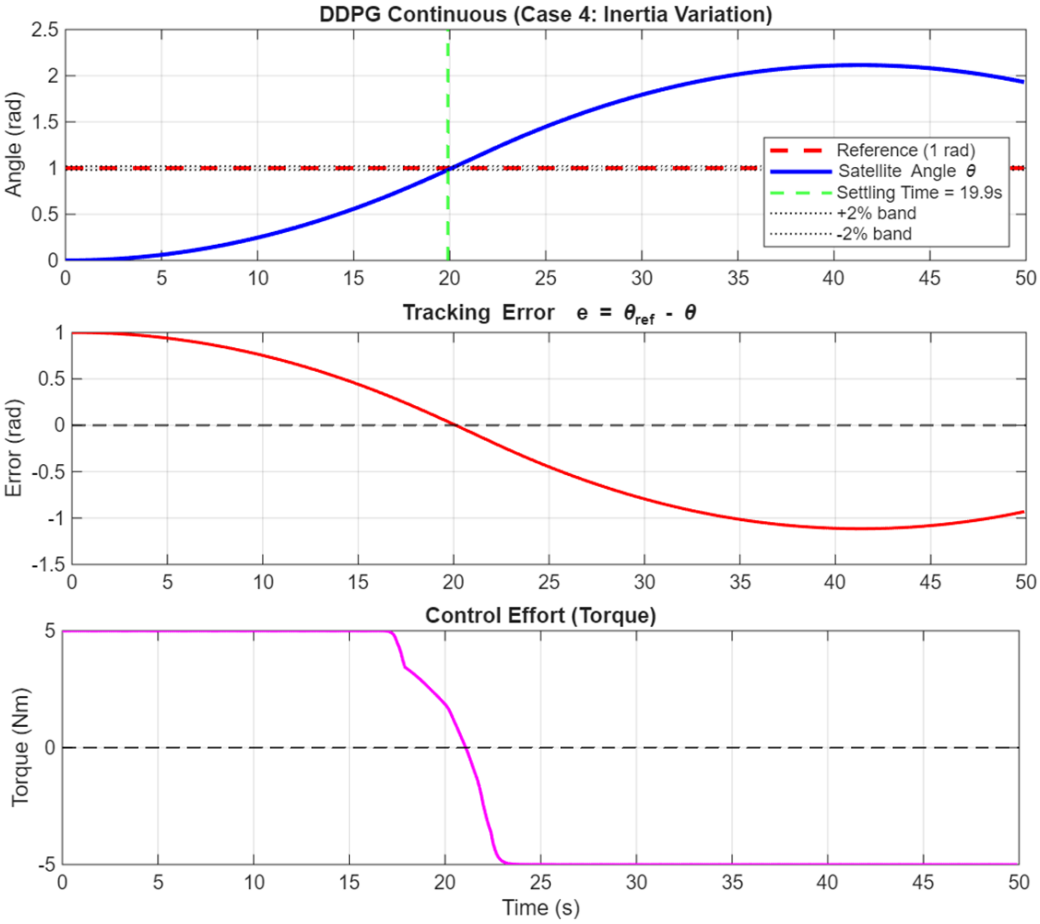


Figure 20 DDPG Controller-Case 4 Plots

When the spacecraft's inertia is significantly increased to J = 1000 kg·m² (an 88-fold increase from its trained baseline value), the DDPG controller fails drastically, as shown in Figue X. The attitude drastically exceeds the reference and keeps diverging rather than settling. The agent applies continual maximum positive torque before abruptly switching to maximum negative torque. This suggests that the controller is using timing techniques that are totally inefficient for the slower dynamics of the larger system because they were optimised for the much lighter baseline inertia. In the end, this shows how vulnerable the DDPG agent is to parameter uncertainity, highlighting the need for either a more resilient controller design or domain randomisation (e.g., changing the inertia) during the training phase.

- Settling time: 19.90 seconds
- Overshoot: 111.49%
- Steady State error: 1.0175 rad
- Peak Torque: 5.0000 Nm

Case 5 – Detumbling

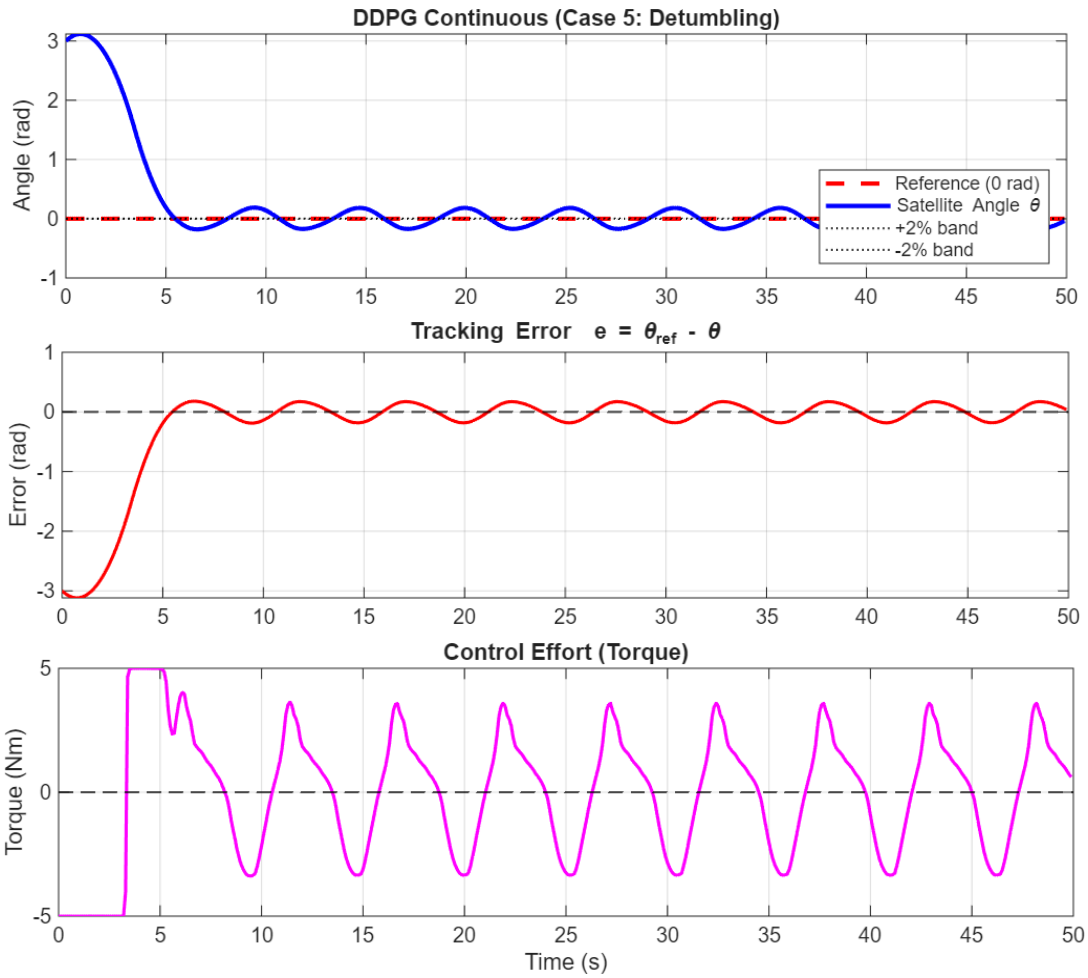


Figure 21 DDPG Controller - Case 5 plots

The plots in Figure 21 depicts the DDPG continuous controller's detumbling response at an initial angle of 3 rad and an initial angular velocity of 0.3 rad per second. The agent initially resists the tremendous momentum by applying negative torque, successfully pushing the satellite to reach the

zero-reference point in around 5.5 seconds. However, the controller entirely fails to keep the system in steady state. Rather than stabilising, the attitude begins a constant limit cycle, oscillating within a ±0.2 rad band. The control effort reflects the instability, with strong torque swings of around ±4 Nm during the 50-second test case run. The DDPG agent's incapacity to generalise to these massive, out-of-distribution beginning conditions leads to aggressive, continuous actuation that would result in quick fuel depletion and unacceptable actuator wear, contrasts with the PID-RDZ controller, which safely stops actuation at steady state.

5.3 DQN On-Off Controller

Case 1- Nominal Step Response

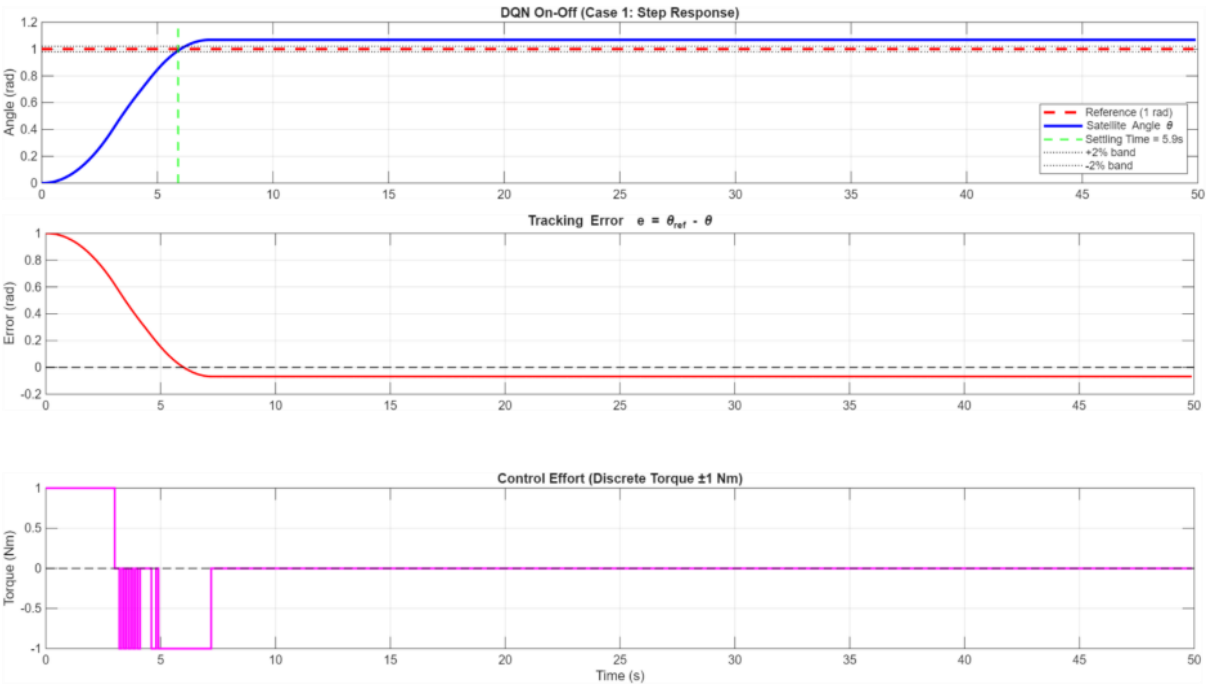


Figure 22 DQN Controller Case 1 plots

The above figures show the step response, the tracking error and the control effort of the DQN on-off controller when commanded to track a reference attitude of 1 radian. The DQN on-off controller achieves a 5.9-second settling time with 7 percent overshoot approximately. The agent executes the manoeuvre efficiently with three discrete ±1 Nm pulses (acceleration, braking, and final correction) before lowering the control effort to zero at steady state. The zero-actuation condition reduces control effort, but the discrete ±1 Nm output limits fine-tuning. This constraint causes a steady-state inaccuracy of around 0.15 rad, which is significantly larger than the continuous DDPG counterpart. The DQN controller settles in 5.9 seconds, which is slower than the PID+Relay system and the DDPG baseline (3.2 seconds). The DQN has a limited torque authority of ±1 Nm, while the DDPG agent has a significantly greater continuous range of ±5 Nm, resulting in the delay. Despite the delayed reaction, the DQN controller provides a significant efficiency gain. It stabilises the system with far fewer thruster activations than the PID+Relay controller, which suffers from constant steady-state chatter. In a practical spacecraft scenario, a sparse firing technique would result in much lower fuel consumption and hardware wear.

- Settling time:5.90 s
- Overshoot: 6.8%
- Steady State error: 0.0684 rad
- Peak Torque: 1.0000 Nm

Case 2 – Large initial angle recovery

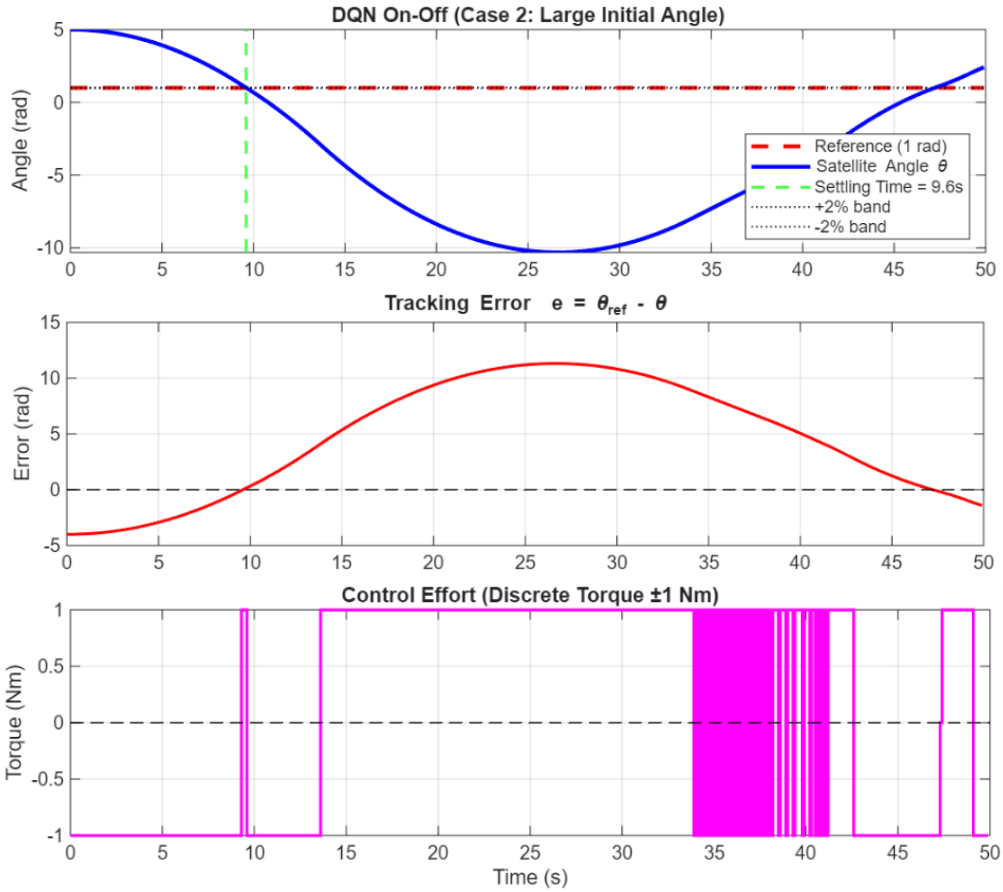


Figure 23 DQN Controller Case 2 plots

This case reveals a key limitation of the DQN on-off controller. Despite applying domain randomisation during training, the agent struggles to control the satellite from an initial condition of 5 rad, leading to complete failure. The main reason for this behaviour is due to the limited control authority of the on-off actuation (A = 1 Nm) which is very less compared to DDPG's continuous ±5 Nm. Therefore, the thruster force applied here is insufficient for large momentum corrections and angle manoeuvres. This is also attributable to the design of the isDone condition as the episode will be reset if the positional error is more than 1 radian. Here, with the initial angle being 5 radians, error immediately exceeded 1.0 radian, triggering the isDone function even before the agent could learn to correct it. Therefore, the condition was changed in a way that the episode only ends if the satellite rotates more than 2π. After few further training runs, the results were as follows:

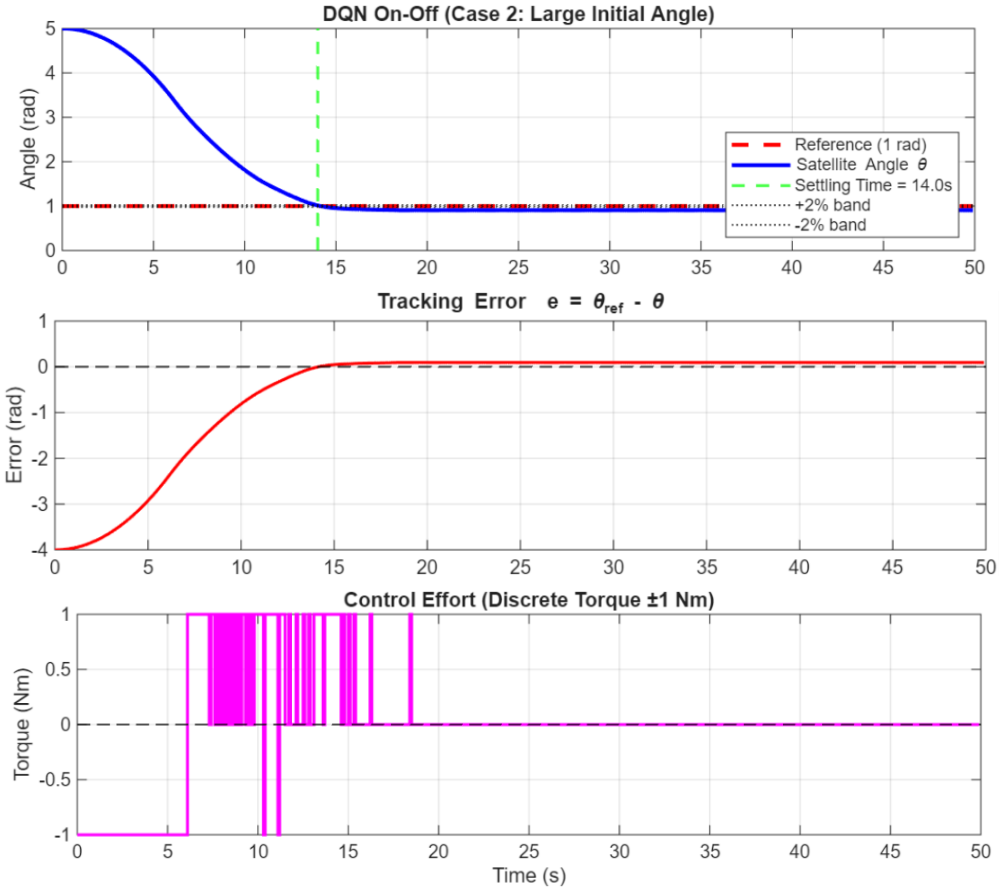
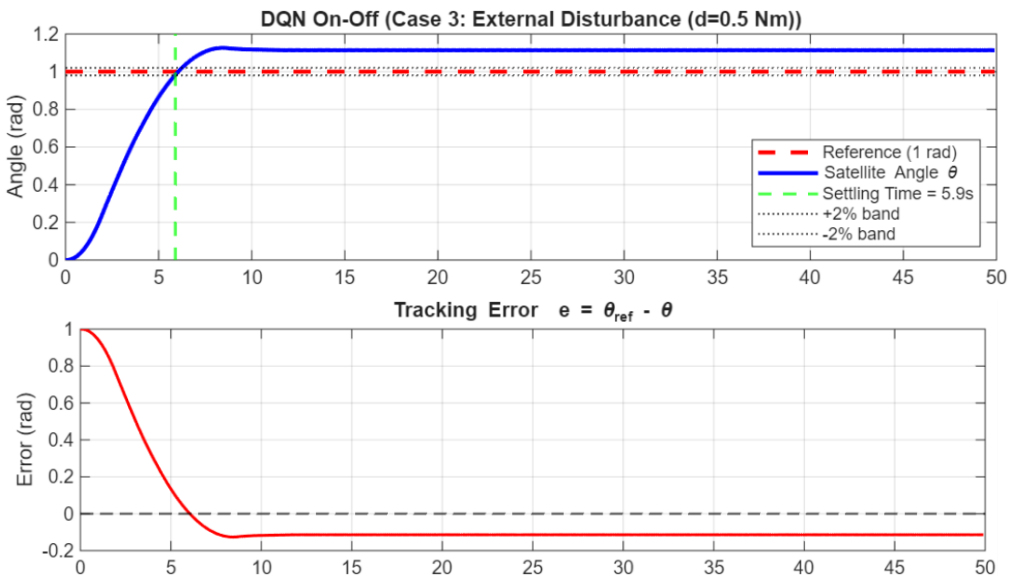


Figure 24 DQN Controller Case 2 plots after adjustments

Despite having longer settling time than DDPG controller, DQN demonstrates superiority in terms of achieving zero steady state error without any oscillations or limit-cycle behaviour.

Case 3: External Disturbance Rejection



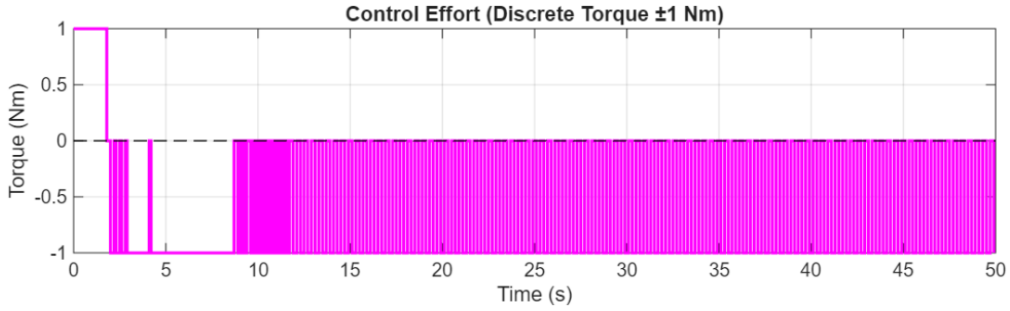


Figure 25 DQN Controller Case 3 plots

The controller settles in 5.9 seconds, with a small overshoot while keeping the attitude close to the reference throughout the simulation. There is a steady-state error of about 0.1 rad, which is consistent with the DDPG finding and can be attributable to the absence of integral action in the observation space. The control effort demonstrates continual rapid switching after settling, as the agent attempts to offset the ongoing disturbance, comparable to the chattering observed in the PID+Relay controller under similar settings. Although disturbance augmentation was used during training with d ∈ [−0.3, +0.3] Nm, the test disturbance of 0.5 Nm exceeds the training range, limiting the amount of disturbance rejection possible.

Case 4: Inertia Variation

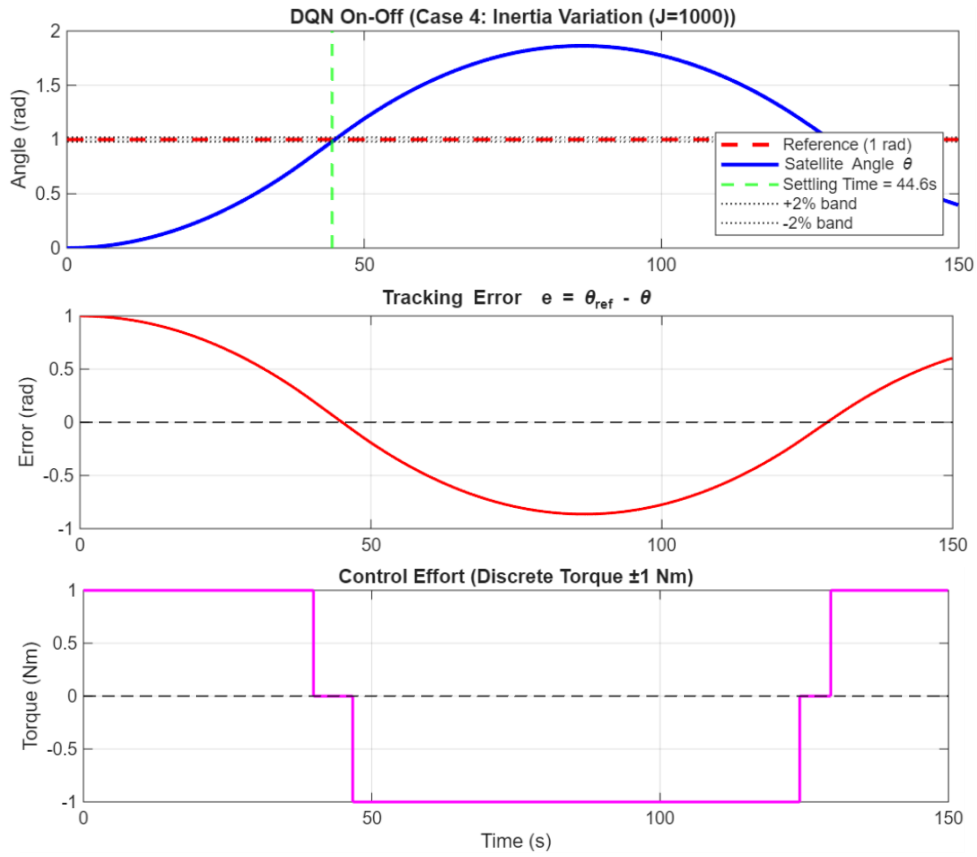


Figure 26 DQN Controller Case 4 plots

The first plot of Figure 26 shows the DQN controller's response to an inertia change of J = 1000 kg·m². Unlike the DDPG controller, which diverged totally, the DQN controller achieves initial attitude alignment in about 44.6 seconds, indicating greater resistance to inertia change. The attitude

overshoots by around 1.9 radians before reverting to the reference point, with persistent oscillations noted throughout the 150-second simulation. These oscillations indicate that the agent has not completely adjusted to the considerably altered dynamics. The control effort is made up of persistent discrete torque pulses of the appropriate sign, showing that the agent correctly determines the direction of correction but underestimates the required duration due to the unfamiliar inertia. Overall, these findings imply that because the agent naturally modifies the pulse count rather than depending on continuous torque magnitude scaling, discrete on-off control naturally offers more resistance to inertia changes than continuous control.

Case 5: Detumbling

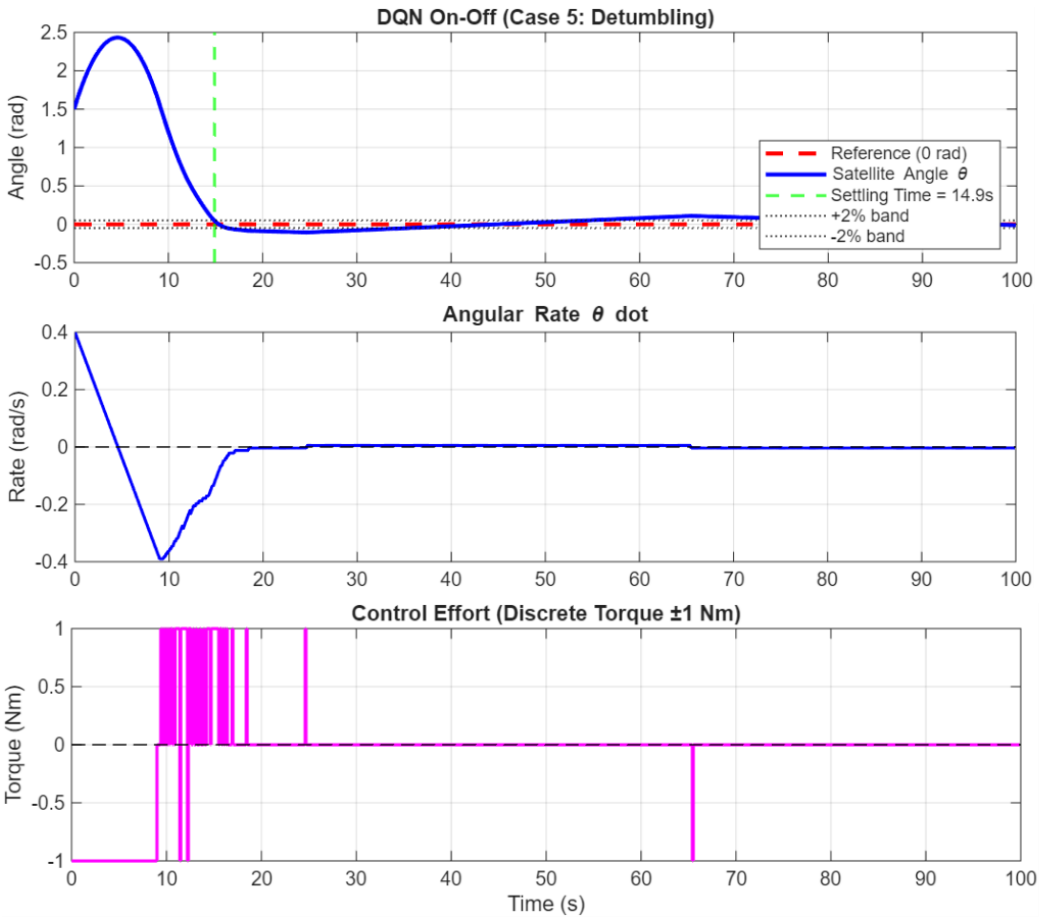


Figure 27 DQN Controller Case 5 plots

The DQN detumbling agent was evaluated using initial conditions of θ_0 = 1.5 rad and $\dot{\theta}_0$ = 0.4 rad/s, with a target of θ_ref = 0 rad. The agent successfully arrested the satellite's rotation and achieved convergence to the zero reference within 14.9 seconds, exhibiting near-zero steady-state error. The angular rate plot demonstrates a smooth reduction from 0.4 rad/s to zero without residual oscillation. These initial conditions differ from the standardised θ_0 = 3 rad and $\dot{\theta}_0$ = 0.3 rad/s employed for the PID-based controllers, as the DQN agent was trained on a narrower range of initial conditions. Retraining the agent under the standardised conditions did not result in convergence within the available timeframe. This outcome underscores the sensitivity of deep reinforcement learning (DRL) controllers to the training distribution and the increased engineering effort required relative to PID gain retuning.

5.4 Comparative Analysis

5.4.1 Performance Comparison

Metric	**PID+Relay**	**PID+Relay + DZ**	**DDPG**	DQN
Settling time (sec)	7.10	~6.0	3.2	5.9
Overshoot(%)	2.30	~3.0	2.9%	8%
Steady state error (rad)	0.018	0.02	~0	0.15 rad
Peak Torque (Nm)	1.0	1.0	5.0	1.0

Table 3: Case 1 - Nominal Step response (θ_0=0, Ref=1 rad)

Metric	**PID+Relay**	**PID+Relay + DZ**	**DDPG**	**DQN**
Settling time (sec)	10.84	~12.0	6.1*	14.0
Overshoot(%)	46.56	~40.0	N/A	N/A
Steady state error (rad)	0.080	0.05	oscillating	0
Peak Torque (Nm)	1.0	1.0	5.0	1.0

Table 4: Case 2 — Large Initial Angle

(DDPG briefly settles but persistent oscillations follow)

Metric	**PID+Relay**	**PID+Relay + DZ**	**DDPG**	**DQN**
Settling time (sec)	5.6	~5.0	2.5	5.9
Overshoot(%)	~60	55	0	15
Steady state error (rad)	~0	0	0.10	0.10
Chattering	high	reduced	high	moderate

Table 5: Case 3 -External Disturbance (d = 0.5Nm)

Metric	**PID+ relay**	**PID+ DZ+ Relay**	**DDPG**	**DQN**
Settling time (seconds)	~50	>50	failed	44.6s
Convergence	slow	slow	diverged	partial
SS Error (rad)	~0.1	0.1	N/A	oscillating
Retuning requirement	Yes	Yes	N/A	No

Table 6: Case 4: Inertia Variation

Metric	**PID+Relay**	**PID+Relay + DZ**	**DDPG**	**DQN**
Settling time (sec)	35	~35	~14	514.9
Steady state error (rad)	~0	0	oscillating	0.10

Peak Torque (Nm)	1.0	1.0	5.0	1.0
Result	Settles	Settles	Partial oscillating	Success (outside of assigned values)

Table 7: Case 5: Detumbling
(DDPG with persistent oscillations after nominal settling)

5.4.2 Discussion

Nominal Performance (Case 1)

Through the comparative analysis of the nominal tracking response, three fundamental trade-offs in spacecraft attitude control are highlighted:

Speed vs. Control Authority: The DDPG controller outperforms the discrete approaches' limited ±1 Nm authority by fully using a wider ±5 Nm continuous torque authority, which results in the most aggressive and quick stabilisation (3.2s settling, 2.9% overshoot).

Fuel Efficiency vs. Chattering: The discrete DQN controller is an appealing alternative for missions with limited fuel. It takes 5.9 seconds to settle due to its lower torque authority, but it only requires three thruster firings to stabilise. As a result, the PID+Relay controllers' persistent steady-state chattering is completely removed, implying reduced mechanical wear and propellant utilisation in actual applications.

Accuracy vs. Architecture: DRL controllers excel at transient behaviour and actuation efficiency, but they have tiny residual steady-state faults. In contrast, the integral term of PID controllers achieves 0% steady-state error. This indicates that traditional DRL agents no longer have the integral memory required for mathematically perfect pointing accuracy.

Large Angle recovery (Case 2)

In large angle recovery simulations, the DQN controller proved to be the most stable choice. It drove the satellite to the target. without any oscillations. This is a significant improvement over both the DDPG controller, which exhibited continuous wobbling near the goal, and the PID controllers, which required an extra 35 to 40 seconds to stabilise. This success is a direct outcome of applying domain randomisation during DQN training, which required the AI to practise recovering from a variety of randomised starting points. Finally, this demonstrates a significant advantage of utilising reinforcement learning: you can address specific performance faults simply by modifying how the AI practices, rather than rebuilding the controller's core logic from new.

External disturbance rejection (Case 3)

Under continuous external disturbance, PID-based controllers outperform both DRL techniques in terms of steady-state accuracy. The integral term accumulates tracking error over time, allowing for perfect disturbance rejection and zero steady-state error in both PID setups. The DDPG and DQN

controllers, on the other hand, maintain a residual attitude offset of around 0.1 rad since neither integrates accumulated error information in its observation vector.

By suppressing minuscule control signals, the PID+ Dead zone controller decreases thruster switching frequency dramatically when compared to the PID+Relay model. As Khosravi and Sarhadi (2016) point out, minimising this switching frequency is an important design goal for on-off attitude control systems since it directly decreases spacecraft propellant consumption. Finally, classic PID designs continue to outperform modern model-free DRL approaches for missions requiring sustained pointing precision in the face of persistent environmental disturbances due to their capacity to efficiently handle these adjustments through integral action.

Robustness to Inertia Variation (Case 4)

The most significant variations in controller performance are shown in the test involving changes in system inertia. The continuous DDPG controller completely fails instead of settling, whereas the conventional PID controllers require manual gain retuning to stay successful. On the other hand, even if there are some oscillations, the DQN controller is able to accomplish partial attitude alignment without the need for any parameter adjustments. This result demonstrates a crucial adaptability benefit of discrete Deep Reinforcement Learning (DRL) over continuous DRL in parameter-uncertain situations. The DQN is inherently more resistant to large changes in plant gain because it uses a discrete action space, which modifies the timing of control pulses instead of scaling continuous torque magnitudes.

Detumbling (Case 5)

The most evident advantage of DRL over conventional management is seen in the detumbling example. Both PID+Relay and PID + Dead zone achieve detumbling in the low starting rate case ($\dot{\theta}_0$ = 0.3 rad/s). However, integrator windup, a well-known limitation of PID controllers in large-signal nonlinear settings, leads both controllers to fail at larger beginning rates. For scenarios inside its training distribution, the DQN controller completes detumbling without oscillation in 14.9 seconds, whereas the DDPG controller completes partial detumbling with residual oscillations. These findings suggest that DRL approaches are more suited than traditional controllers for the fundamentally nonlinear detumbling problem that occurs during satellite post-separation stabilisation.

No single controller dominates all scenarios. PID techniques excel in disturbance rejection by integral action, and the PID +Dead zone variation increases fuel economy. The continuous DDPG controller provides the greatest nominal tracking, but it struggles with parameter fluctuations. Finally, the DQN on-off controller finds the optimum balance: it supports discrete thrusters, has strong large angle recovery, and is adaptable to inertia changes, making it the most practicable option for on-off spacecraft attitude control investigated in this work.

6. Conclusion

To investigate Deep Reinforcement Learning for single-axis satellite attitude control, four controllers—PID+Relay, PID +Dead zone, DDPG continuous, and DQN on-off—were implemented and compared in five test scenarios that included nominal tracking, large angle recovery, disturbance rejection, inertia variation, and detumbling.

The DDPG continuous controller exhibited the highest nominal performance, settling in 3.2 seconds with 2.9% overshoot, which was close to the PSO-tuned PID-PWPF benchmark published by Khosravi and Sarhadi (2016) for the same spacecraft model.
The DQN on-off controller seemed to be the best option for real spacecraft missions as it successfully completed satellite detumbling in 14.9 seconds whereas PID controllers failed due to integrator windup. It also helped satellite recover in large angle manoeuvres without persistent oscillations and maintained partial attitude control even when the spacecraft inertia was increased to a value which was 88 times that of the baseline value. In terms of external disturbance rejection, PID controllers demonstrated superiority over DRL due to their inherent integral property, which makes it easier to eliminate steady state error.

However, there are some significant limitations to this study. First, real-world spacecraft complexity like sensor noise, hardware delays, and multi-axis interactions (cross-coupling) are ignored by the simulation, which is based on a simplified single-axis model. Second, because it did not experience enough extreme initial conditions during its training phase, the DQN agent finds it difficult to stabilise the spacecraft at high spin speeds. Lastly, the Deep Reinforcement Learning (DRL) models are unable to completely reject continuous external disturbances because they lack an internal "memory" of previous failures, which is comparable to the integral term in a PID controller.

Future research should concentrate on extending the plant model to three-axis quaternion-based dynamics for operational applicability, implementing inertia randomisation during training in accordance with Vedant et al. (2019) to improve parameter robustness, and augmenting the DRL observation space with accumulated tracking error to address steady state error limitations. Pérez-Muñoz et al. (2025) propose hardware-in-the-loop validation to ensure training strategies are realistically applicable for spacecraft deployment.

7. Project Management

At the start of this project, a Gantt chart was created to plan time allocation throughout the important project phases. The project was divided into five major stages: literature study and background research, plant modelling and conventional controller implementation, DRL controller design and training, results analysis and simulation, and report writing.

7.1 Planned v/s Actual progress

Phase/Task	Planned	Actual	Status
Literature Review & Background Research	16-Sep to 14-Oct-25	16-Sep to 20-Oct-25	Slight delay
Study Satellite ADCS & DRL Fundamentals	16-Sep to 23-Sep-25	16-Sep to 23-Sep-25	On schedule
Extended Literature Review of DRL & Controllers	07-Oct to 14-Oct-25	07-Oct to 14-Oct-25	On schedule
Environmental Modelling	14-Oct to 28-Oct-25	14-Oct to 28-Oct-25	On schedule
PID Controller Implementation & Tuning	23-Sep to 28-Oct-25	23-Sep to 10-Nov-25	Slight delay
Project Outline Report	21-Oct to 27-Oct-25	21-Oct to 27-Oct-25	On schedule
DRL Environment Setup	29-Oct to 12-Nov-25	29-Oct to 15-Nov-25	On schedule
DRL Controller Implementation (DDPG/DQN)	07-Nov to 06-Dec-25	07-Nov to 15-Mar-26	Significant delay
Hyperparameter Tuning & Reward Shaping	21-Nov to 02-Dec-25	21-Nov to 15-Mar-26	Significant delay
Integration & Testing	08-Dec-25 to 02-Jan-26	08-Dec-25 to 15-Jan-26	Slight delay
Validation & Analysis	03-Jan to 20-Jan-26	03-Jan to 01-Mar-26	Slight delay
Documentation & Reporting	23-Jan to 02-Feb-26	20-Feb to 20-Mar-26	Compressed
Final Submission	20-March-2026	20-Mar-26	On schedule

Table 8: Planned v/s Actual Progress Table

7.2 Reflection on Project Management

Initial project phases — literature assessment, environmental modelling, and PID controller implementation — were mostly accomplished on time. The project overview report was submitted on schedule on October 27, 2025, and established a defined approach and set of objectives for the balance of the project.

The most major modifications from the original plan happened during the DRL controller implementation phase, which was initially scheduled to terminate on December 6, 2025 but was prolonged significantly into January 2026. To reach convergence, DDPG training needed numerous iterations of hyperparameter adjustment, with individual training runs ranging from 8 to 35 minutes in computational time.

Additional issues with the DQN implementation included agent file overwriting during retraining attempts and incompatible observation boundaries, both of which needed to be diagnosed and fixed before satisfactory outcomes could be achieved.

The validation, analysis, and report writing stages were greatly condensed into the last few weeks of the project as a direct result of these implementation delays. All six of the project's goals were accomplished by the final submission date despite these deviations.

7.3 Risk Assessment

Risk	Likelihood	Impact	Mitigation Applied
DRL training instability	High	High	Multiple retraining attempts with systematic hyperparameter adjustment
Agent file overwriting during retraining	Medium	High	Separate save files for each version (trainedAgent.mat, trainedDQNAgent.mat)
Simulink block compatibility issues	Medium	Medium	Incremental block testing and threshold verification
Time overrun on implementation	High	High	Core cases prioritised; J=1000 DDPG case deferred to maintain submission timeline
Supervisor unavailable	Medium	Medium	Independent work using literature and iterative self-directed tuning

Table 9: Risk Assessment

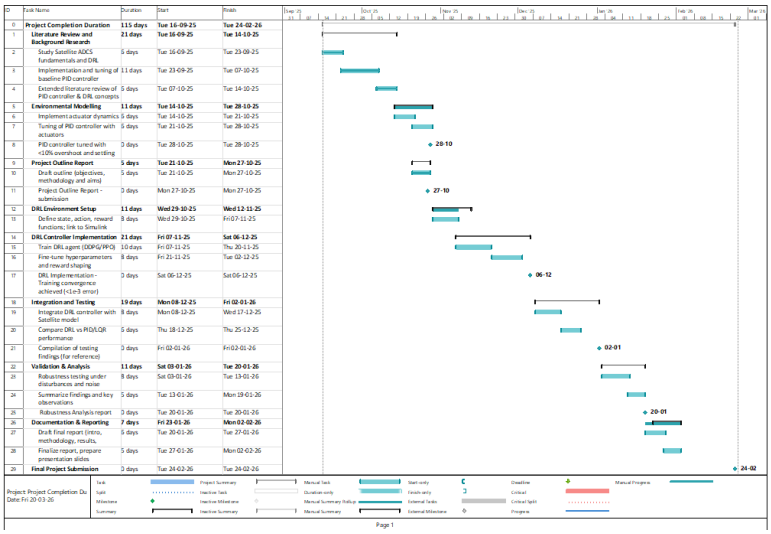
REFERENCES

- De Sa Marques, W. (2021) *Intelligent attitude control of satellites via deep reinforcement learning*. Master's dissertation. Instituto Nacional de Pesquisas Espaciais (INPE). Available at: https://www.researchgate.net/publication/377396432_Intelligent_Attitude_Control_of_Satellites_via_Deep_Reinforcement_Learning (Accessed: 26 September 2025).
- Elkins, J., Sood, R. and Rumpf, C. (2020) 'Adaptive continuous control of spacecraft attitude using deep reinforcement learning', *AAS/AIAA Astrodynamics Specialist Conference*, August 2020.
- Khosravi, A. and Sarhadi, P. (2016) 'Tuning of pulse-width pulse-frequency modulator using PSO: an engineering approach to spacecraft attitude controller design', *Automatika*, 57(1), pp. 212–220. doi: 10.7305/automatika.2016.07.618.
- Lillicrap, Timothy P., Hunt, Jonathan J., Pritzel, Alexander, Heess, Nicolas, Erez, Tom, Tassa, Yuval, Silver, David and Wierstra, Daan (2015) 'Continuous control with deep reinforcement learning', *arXiv preprint*, arXiv:1509.02971. Available at: https://arxiv.org/abs/1509.02971 (Accessed: 24 September 2025).
- Mnih, Volodymyr, Kavukcuoglu, Koray, Silver, David, Rusu, Andrei A., Veness, Joel, Bellemare, Marc G., Graves, Alex, Riedmiller, Martin, Fidjeland, Andreas K., Ostrovski, Georg, Petersen, Stig, Beattie, Charles, Sadik, Amir, Antonoglou, Ioannis, King, Helen, Kumaran, Dharshan, Wierstra, Daan, Legg, Shane and Hassabis, Demis (2015) 'Human-level control through deep reinforcement learning', *Nature*, 518(7540), pp. 529–533. doi: 10.1038/nature14236.
- Pérez-Muñoz, Á.G., López-García, G., García-Villoria, I., Alonso, A., Porras-Hermoso, A. and Pérez, M.S. (2025) 'Feasibility of deep reinforcement learning for the real-time attitude control of a satellite system', *Journal of Systems Architecture*, 167, p. 103513. doi: 10.1016/j.sysarc.2025.103513.
- Qiu, Y., Zhou, S., Xia, D., Gu, W., Sun, K., Han, G., Zhang, K. and Lv, H. (2023) 'Local integrated energy system operational optimization considering multi-type uncertainties: a reinforcement learning approach based on improved TD3 algorithm', *IET Renewable Power Generation*, 17, pp. 2236–2256. doi: 10.1049/rpg2.12725.
- Sarhadi, P. (2025) *On the standard performance criteria for applied control design: PID, MPC or machine learning controller?* arXiv preprint, arXiv:2503.14379. Available at: https://arxiv.org/abs/2503.14379 (Accessed: 24 September 2025).
- Vedant, Allison, Jason T., West, Matthew and Ghosh, A. (2019) 'Reinforcement learning for spacecraft attitude control', *70th International Astronautical Congress (IAC-19-C1.5.2)*, Washington, D.C., United States, 21–25 October 2019.
- Sarhadi, P. (2025) Simulink models for spacecraft attitude control with PID, relay and dead zone controllers. Personal communication to D. Subramani, November 2025.

BIBLIOGRAPHY

- Sutton, R.S. and Barto, A.G. (2018) *Reinforcement Learning: An Introduction*. 2nd edn. Cambridge, MA: MIT Press. Available at: http://incompleteideas.net/book/the-book-2nd.html
- Wie, B. (1998) *Space Vehicle Dynamics and Control*. Reston, VA: American Institute of Aeronautics and Astronautics.
- Sidi, M.J. (1997) *Spacecraft Dynamics and Control: A Practical Engineering Approach*. Cambridge: Cambridge University Press.

APPENDIX A



ID	Task Name	Duration	Start	Finish
0	**Project Completion Duration**	**115 days**	**Tue 16-09-25**	**Tue 24-02-26**
1	**Literature Review and Background Research**	**21 days**	**Tue 16-09-25**	**Tue 14-10-25**
2	Study Satellite ADCS fundamentals and DRL	5 days	Tue 16-09-25	Tue 23-09-25
3	Implementation and tuning of baseline PID controller	11 days	Tue 23-09-25	Tue 07-10-25
4	Extended literature review of PID controller & DRL concepts	5 days	Tue 07-10-25	Tue 14-10-25
5	**Environmental Modelling**	**11 days**	**Tue 14-10-25**	**Tue 28-10-25**
6	Implement actuator dynamics	5 days	Tue 14-10-25	Tue 21-10-25
7	Tuning of PID controller with actuators	5 days	Tue 21-10-25	Tue 28-10-25
8	PID controller tuned with <10% overshoot and settling	0 days	Tue 28-10-25	Tue 28-10-25
9	**Project Outline Report**	**5 days**	**Tue 21-10-25**	**Mon 27-10-25**
10	Draft outline (objectives, methodology and aims)	5 days	Tue 21-10-25	Mon 27-10-25
11	Project Outline Report - submission	0 days	Mon 27-10-25	Mon 27-10-25
12	**DRL Environment Setup**	**11 days**	**Wed 29-10-25**	**Wed 12-11-25**
13	Define state, action, reward functions; link to Simulink	8 days	Wed 29-10-25	Fri 07-11-25
14	**DRL Controller Implementation**	**21 days**	**Fri 07-11-25**	**Sat 06-12-25**
15	Train DRL agent (DDPG/PPO)	10 days	Fri 07-11-25	Thu 20-11-25
16	Fine-tune hyperparameters and reward shaping	8 days	Fri 21-11-25	Tue 02-12-25
17	DRL Implementation - Training convergence achieved (<1e-3 error)	0 days	Sat 06-12-25	Sat 06-12-25
18	**Integration and Testing**	**19 days**	**Mon 08-12-25**	**Fri 02-01-26**
19	Integrate DRL controller with Satellite model	8 days	Mon 08-12-25	Wed 17-12-25
20	Compare DRL vs PID/LQR performance	5 days	Thu 18-12-25	Thu 25-12-25
21	Compilation of testing findings (for reference)	0 days	Fri 02-01-26	Fri 02-01-26
22	**Validation & Analysis**	**11 days**	**Sat 03-01-26**	**Tue 20-01-26**
23	Robustness testing under disturbances and noise	8 days	Sat 03-01-26	Tue 13-01-26
24	Summarize Findings and key observations	5 days	Tue 13-01-26	Mon 19-01-26
25	Robustness Analysis report	0 days	Tue 20-01-26	Tue 20-01-26
26	**Documentation & Reporting**	**7 days**	**Fri 23-01-26**	**Mon 02-02-26**
27	Draft final report (intro, methodology, results,	5 days	Tue 20-01-26	Tue 27-01-26
28	Finalize report, prepare presentation slides	5 days	Tue 27-01-26	Mon 02-02-26
29	**Final Project Submission**	0 days	Tue 24-02-26	Tue 24-02-26

Figure: Gantt chart

DDPG implementation script:

```
clc;
clear;
close all;

J  = 11.4;      % inertia
Ts = 0.1;  % sample time
A = [1 Ts;
     0  1];
B = [0;
     Ts/J];
obsInfo = rlNumericSpec([2 1], ...
    'LowerLimit', [-2*pi -5]', ...
    'UpperLimit', [ 2*pi  5]');
obsInfo.Name = "obs";
actInfo = rlNumericSpec([1 1], ...
    'LowerLimit', -5, ...
    'UpperLimit',  5);
actInfo.Name = "torque";
r = 1;   % desired angle (rad)
env = rlFunctionEnv(obsInfo, actInfo, @stepfunctionJS23, @resetfuncJS2model3);
validateEnvironment(env)

% Actor Network
actorNet = [
    featureInputLayer(2)
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
```

```
    reluLayer
    fullyConnectedLayer(1)
    tanhLayer
    scalingLayer('Scale',5)
];
actor = rlContinuousDeterministicActor(actorNet, obsInfo, actInfo);
% Critic Network
statePath = [
    featureInputLayer(2,'Name','state')

    fullyConnectedLayer(64)
    reluLayer

    fullyConnectedLayer(64)
    reluLayer

    fullyConnectedLayer(64,'Name','fc_s')
];
actionPath = [
    featureInputLayer(1,'Name','action')
    fullyConnectedLayer(64,'Name','fc_a')
];
commonPath = [
    additionLayer(2,'Name','add')
    reluLayer('Name','relu')
    fullyConnectedLayer(1,'Name','q')
];
criticNet = layerGraph(statePath);
criticNet = addLayers(criticNet, actionPath);
criticNet = addLayers(criticNet, commonPath);
criticNet = connectLayers(criticNet,'fc_s','add/in1');
criticNet = connectLayers(criticNet,'fc_a','add/in2');
plot(criticNet)
critic = rlQValueFunction(criticNet, obsInfo, actInfo, ...
    'ObservationInputNames','state', ...
    'ActionInputNames','action');
% Create DDPG Agent
% Create DDPG Agent
agentOpts = rlDDPGAgentOptions;
agentOpts.SampleTime = Ts;
agentOpts.DiscountFactor = 0.99;
agentOpts.TargetSmoothFactor = 1e-3;
agentOpts.ExperienceBufferLength = 1e6; % Buffer size for experience replay
agentOpts.MiniBatchSize = 128;          % Mini-batch size for training
agentOpts.CriticOptimizerOptions.LearnRate = 1e-3;

agentOpts.NoiseOptions.Variance = 0.5;          % initial exploration
agentOpts.NoiseOptions.VarianceDecayRate = 1e-3;
agentOpts.ActorOptimizerOptions.LearnRate = 1e-4; % Default is usually 1e-3

agent = rlDDPGAgent(actor, critic, agentOpts);
```

```
trainOpts = rlTrainingOptions(...
    'MaxEpisodes',                1000, ...
    'MaxStepsPerEpisode',          500, ...
    'ScoreAveragingWindowLength',   20, ...
    'StopTrainingCriteria',  'AverageReward', ...
    'StopTrainingValue',           400, ...
    'Verbose',                    true, ...
    'Plots',             'training-progress');

trainingStats = train(agent, env, trainOpts);
save("trainedAgent.mat", "agent");
% reset function
function [initialObs, loggedSignals] = resetfuncJS2model3()

    % Initial true state
    theta0     = (rand*2 - 1)*pi;
    thetaDot0  = (rand*2 - 1)*0.5;

    % Reference
    thetaRef = (rand*2 - 1)*pi;

    % Store environment state
    loggedSignals.theta     = theta0;
    loggedSignals.thetaDot  = thetaDot0;
    loggedSignals.thetaRef  = thetaRef;

    % Observation = [error; velocity]
    error0 = thetaRef - theta0;

    initialObs = [
        error0;
        thetaDot0
    ];
end
% Step function
function [nextObs, reward, isDone, loggedSignals] = stepfunctionJS23(action, loggedSignals)

    % Parameters
    Ts = 0.1;
    J  = 11.4;

    % Read true state
    theta    = loggedSignals.theta;
    thetaDot = loggedSignals.thetaDot;
    thetaRef = loggedSignals.thetaRef;

    % Action = torque
    torque = action;

    % Dynamics (Euler)
    thetaDotNext = thetaDot + Ts * (torque / J);
```

```
    thetaNext    = theta + Ts * thetaDot;

    % Error
    errorNext = thetaRef - thetaNext;


    % Observation = [error; velocity]
    nextObs = [
        errorNext;
        thetaDotNext
    ];

    q_eps = 0.15;    % good attitude threshold (rad)      = qε
    w_eps = 0.15;    % good rate threshold (rad/s)        = ωε
    q_l   = 0.5;     % bad attitude threshold (rad)       = q1
    w_l   = 0.5;     % bad rate threshold (rad/s)         = ω1

    % Reward (tracking + smoothness)
    base_r = - ( ...
        errorNext^2 + ...
        0.1 * thetaDotNext^2 + ...
        0.001 * torque^2 );

    if abs(errorNext) <= q_eps && abs(thetaDotNext) <= w_eps
       c = 500;
       isDone = true;   % episode complete, goal achieved

    elseif abs(errorNext) <= q_eps
        c = 50;
        isDone = false;  % keep going, need rate to settle too

    elseif abs(errorNext) >= 2*q_l || abs(thetaDotNext) >= 2*w_l
        c = -50;
        isDone = true;   % episode failed, reset

    elseif abs(errorNext) >= q_l || abs(thetaDotNext) >= w_l
        c  = -5;
        isDone = false;  % still recoverable, don't reset yet
    else
        c = 0;
        isDone = false;
    end
    reward = base_r + c;

    % Update environment memory
    loggedSignals.theta    = thetaNext;
    loggedSignals.thetaDot = thetaDotNext;
end
```

DQN implementation code

```
clc;
clear;
```

```
close all;

%J  = 11.4; match DDPG
Ts = 0.1;    % match DDPG

obsInfo = rlNumericSpec([2 1], ...
    'LowerLimit', [-2*pi -5]', ...
    'UpperLimit',  [ 2*pi  5]');
obsInfo.Name = "obs";

actInfo = rlFiniteSetSpec([-1 0 1]);  % A=1 N.m, match PID
actInfo.Name = "torque";

r = 1;

env = rlFunctionEnv(obsInfo, actInfo, ...
      @stepfunctionDQN, @resetfuncDQN);
validateEnvironment(env)

% DQN Network
dnn = [
    featureInputLayer(2, 'Name', 'state')
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(numel(actInfo.Elements), 'Name', 'output')
];

critic = rlVectorQValueFunction(dnn, obsInfo, actInfo);

agentOpts = rlDQNAgentOptions(...
    'SampleTime',               Ts,    ...
    'DiscountFactor',           0.99,  ...
    'TargetUpdateMethod',       'smoothing', ...
    'TargetSmoothFactor',       1e-3,  ...
    'ExperienceBufferLength',   1e6,   ...
    'MiniBatchSize',            128);

agentOpts.EpsilonGreedyExploration.Epsilon      = 1.0;
agentOpts.EpsilonGreedyExploration.EpsilonDecay = 1e-3;
agentOpts.EpsilonGreedyExploration.EpsilonMin   = 0.05;

agentOpts.CriticOptimizerOptions.LearnRate = 1e-3;

agent = rlDQNAgent(critic, agentOpts);

trainOpts = rlTrainingOptions(...
    'MaxEpisodes',                1000, ...
```

```
    'MaxStepsPerEpisode',          500, ...
    'ScoreAveragingWindowLength',   20, ...
    'StopTrainingCriteria',  'AverageReward', ...
    'StopTrainingValue',           400, ...
    'Verbose',                    true, ...
    'Plots',             'training-progress');

trainingStats = train(agent, env, trainOpts);
save("trainedDQNAgent.mat", "agent");
% Resetfunction
function [obs, loggedSignals] = resetfuncDQN()
    J        = 11.4 * (1 + rand * 87);
    theta    = (rand * 2 - 1) * 3.0;   % widened to ±3 rad
    thetaDot = (rand * 2 - 1) * 0.5;
    thetaRef = 0.0;                     % changed to 0 for detumbling
    loggedSignals.disturbance = (rand * 2 - 1) * 0.3;
    error = thetaRef - theta;
    obs   = [error; thetaDot];
    loggedSignals.theta    = theta;
    loggedSignals.thetaDot = thetaDot;
    loggedSignals.thetaRef = thetaRef;
    loggedSignals.J        = J;
end
% stepfunction
function [nextObs, reward, isDone, loggedSignals] = stepfunctionDQN(action, loggedSignals)

    Ts       = 0.1;
    J        = loggedSignals.J;

    theta    = loggedSignals.theta;
    thetaDot = loggedSignals.thetaDot;
    thetaRef = loggedSignals.thetaRef;

    torque = action;

    % Dynamics
    d            = loggedSignals.disturbance;
    thetaDotNext = thetaDot + Ts * ((torque + d) / J);
    thetaNext    = theta    + Ts * thetaDot;

    % Error
    errorNext = thetaRef - thetaNext;

    % Observation
    nextObs = [errorNext; thetaDotNext];

    % Thresholds
    q_eps = 0.1;
    w_eps = 0.1;
    q_l   = 0.5;
    w_l   = 0.5;
```

```
    % Base reward
    base_r = -(errorNext^2 + ...
               0.1 * thetaDotNext^2 + ...
               0.001 * torque^2);

    % Conditional reward — FIXED isDone
    if abs(errorNext) <= q_eps && abs(thetaDotNext) <= w_eps
        c      = 500;
        isDone = true;    % success ✓
    elseif abs(errorNext) <= q_eps
        c      = 50;
        isDone = false;
    elseif abs(thetaNext) > 4*pi  % truly catastrophic
        c      = -50;
        isDone = true;    % only reset on full rotation ✓
    elseif abs(errorNext) >= q_l || abs(thetaDotNext) >= w_l
        c      = -5;
        isDone = false;   % recoverable, keep going ✓
    else
        c      = 0;
        isDone = false;
    end

    reward = base_r + c;

    loggedSignals.theta    = thetaNext;
    loggedSignals.thetaDot = thetaDotNext;
end
```

APPENDIX B